석사학위논문

딥러닝 모델 경량화 기술을 통한 모델 성능 개선과 SDiT 모델 기술 연구

Improving Model Performance through Deep Learning Model Lightweighting Techniques and Research on the SDiT Model Technology.

홍익대학교 대학원

소프트웨어융합학과

최 진 욱

2023년 8월

딥러닝 모델 경량화 기술을 통한 모델 성능 개선과 SDiT 모델 기술 연구

Improving Model Performance through Deep Learning Model Lightweighting Techniques and Research on the SDiT Model Technology.

지도교수 정은성

이 논문을 석사학위 논문으로 제출함.

2023년 8월

홍익대학교 대학원

소프트웨어융합학과

최 진 욱

최진욱의 석사학위 논문을 인준함

심 사 위 원

심사위원장　　유 동 영　　(인)

심사위원　　정 은 성　　(인)

심사위원　　정 성 욱　　(인)

홍익대학교 대학원

국문초록

최근 몇 년 동안, 인공지능 및 딥러닝 분야에서는 많은 데이터와 강력한 컴퓨팅 성능을 활용한 모델의 발전이 지속적으로 이루어지고 있다. 또한, 빅 데이터의 축적과 하드웨어의 발전은 딥러닝 모델의 규모를 키우고 복잡성을 증가시키는 데 기여하였다. 이는 높은 성능과 정확도를 제공하지만, 동시에 모델의 크기와 계산 요구 사항도 크게 증가시키는 문제를 야기했다.

이러한 문제를 해결하기 위해 모델 압축 기법이 중요한 연구 주제로 떠오르게 되었으며, 모델 압축은 딥러닝 모델을 보다 작고 경량화된 형태로 변환하여, 모델 크기를 줄이고 추론 속도를 향상시키는 것을 목표로 연구된다. 이를 통해 제한된 자원을 가진 환경에서도 효율적인 딥러닝 실행을 가능하게 한다. 본 연구에서는 모델 압축 기법인 지식 증류(Knowledge Distillation), 가중치 가지치기(Weight Pruning), 양자화(Quantization)와 같은 대표적인 모델 압축 기법을 분석하고 비교한다. 이 중에서도 Teacher 모델과 Student 모델 간의 지식 전달을 통해 작은 크기의 모델에도 Teacher 모델의 정확도와 특성을 전달하는 방법인 지식 증류 기법이 가장 우수한 성능을 보여주었다.

또한, 최근에는 컴퓨터 비전 분야에서 기존의 컨볼루션 레이어를 대체하는 Transformer 기반의 모델인 ViT(Vision Transformer)가 주목을 받고 있으며, ViT 모델은 Transformer 아키텍처를 활용하여 이미지 데이터를 처리하며, 뛰어난 성능을 보여주고 있다. 그러나 ViT 모델은 많은 파라미터와 높은 데이터 요구량으로 인해 모델 크기와 학습 비용이 증가하는 단점을 지닌다.

따라서, 본 연구에서는 ViT 모델의 단점을 극복하고 적은 데이터로도 높은 성능을 달성할 수 있는 새로운 모델인 SDiT(Self-Knowledge Distillation image Transformer)을 제안한다.

목 차

국문초록 ····· i

목 차 ····· ii

그림목차 ····· iv

표 목차 ····· v

제 1 장 서론 ····· 1

1.1 연구배경 및 목적 ····· 1

1.2 연구내용 및 구성 ····· 2

제 2 장 관련 연구 ····· 4

2.1 모델 경량화 기술 ····· 4

2.1.1 지식 증류 ····· 4

2.1.2 가중치 가지치기 ····· 8

2.1.3 양자화 ····· 9

2.2 딥러닝 모델 ····· 11

2.2.1 VGG ····· 11

2.2.2 ResNet ····· 12

2.2.3 MobileNet ····· 14

2.2.4 EfficientNet ····· 15

2.2.5 Transformer and ViT ····· 16

2.2.6 DeiT ····· 19

제 3 장 모델 경량화 기술 분석 ····· 20
3.1 데이터 소개 ····· 21
3.2 실험 환경 ····· 21
3.3 실험 결과 및 분석 ····· 23

제 4 장 데이터셋 ····· 33
4.1 Pothole 데이터 소개 ····· 33
4.2 데이터 전처리와 데이터 증분 ····· 34

제 5 장 SDiT 모델 제안 및 Hyperparameter 튜닝 ····· 36
5.1 SDiT ····· 37
5.2 Hyperparameter 튜닝 ····· 39

제 6 장 SDiT 실험 환경 및 실험 결과 분석 ····· 39
6.1 실험 환경 ····· 39
6.2 실험 결과 및 향후 연구 ····· 40

제 7 장 결론 및 향후 연구 ····· 45
7.1 향후 연구 ····· 39
7.2 결론 ····· 40

참 고 문 헌 ····· 47
영문요약(ABSTRACT) ····· 51

그림목차

그림 2-1 G.Hinton의 지식 증류 구조 ····· 5
그림 2-2 자기 지식 증류 네트워크 아키텍처 ····· 7
그림 2-3 가중치 가지치기 도식화 ····· 8
그림 2-4 딥러닝 모델의 가중치 가지치기 순서도 ····· 9
그림 2-5 고정 소수점 변수를 정수형 변수로 변환 ····· 10
그림 2-6 VGG16 모델 아키텍처 ····· 12
그림 2-7 Residual Block 구조 ····· 13
그림 2-8 Depthwise Separable Convolution과 Pointwise Convolution 구조 ····· 14
그림 2-9 Transformer 모델 구조 ····· 17
그림 2-10 Vision Transformer(ViT) 모델 구조 ····· 18
그림 2-11 DeiT(Data-efficient image Transformer) 모델 구조 ····· 19
그림 3-1 GPU Server에서의 정확도 비교 ····· 25
그림 3-2 GPU Server에서의 이미지 당 처리시간 비교 ····· 26
그림 3-3 GPU Server에서의 모델 파일사이즈 비교 ····· 27
그림 3-4 Edge Device에서의 정확도 비교 ····· 28
그림 3-5 Edge Device에서의 이미지 당 처리시간 비교 ····· 29
그림 3-6 Edge Device에서의 모델 파일사이즈 비교 ····· 30
그림 4-1 Pothole 데이터의 증분 작업 도식화 ····· 35
그림 5-1 Self-Knowledge Distillation image Transformer(SDiT) 모델 구조 ····· 37
그림 7-1 SDiT 모델의 연합학습 적용 아키텍처 ····· 45

표 목차

표 3-1 ImageNet, CIFAR-10 데이터셋 정보 ····· 21
표 3-2 GPU 서버 스펙 시트 ····· 22
표 3-3 NVIDIA Jetson Nano 스펙 시트 ····· 23
표 3-4 VGG19의 GPU서버에서의 실험 결과 표 ····· 24
표 3-5 ResNet50의 GPU서버에서의 실험 결과 표 ····· 25
표 3-6 VGG19의 Edge Device에서의 실험 결과 표 ····· 27
표 3-7 ResNet50의 Edge Device에서의 실험 결과 표 ····· 28
표 3-8 Case에 따른 가중치 ····· 31
표 3-9 GPU Server에서의 점수 ····· 32
표 3-10 Edge Device에서의 점수 ····· 32
표 4-1 Pothole 데이터셋 정보 ····· 34
표 4-2 데이터 전처리 및 증분 후의 Pothole 데이터셋 정보 ····· 36
표 5-1 GPU Server 실험 환경 스펙 시트 ····· 40
표 6-1 데이터 증분 실험 결과 표 ····· 41
표 6-2 손실 함수 비교 실험 결과 표 ····· 42
표 6-3 다른 모델간의 비교 결과 표 ····· 44

제 1 장 서론

1.1 연구배경 및 목적

강력한 GPU 기반의 컴퓨팅 자원을 바탕으로, 우수한 성능의 딥러닝 모델이 대거 출현하였다. 하지만 우수한 모델은 수많은 파라미터로 구성되어 있으며, 파라미터의 수의 크기만큼 GPU 메모리와 같은 컴퓨팅 리소스가 필요하다 [1]. 모델이 추론 할 때, 대량의 컴퓨팅 리소스가 보장되는 서버 및 데이터 센터의 환경에서는 문제가 발생하지 않지만 컴퓨팅 리소스가 충분하지 않은 IoT/Edge Device 환경에 적용하기 위해 모델 압축(Model Compression) 기법의 필요성 또한 증가하고 있다.

또한, 최근 몇 년동안 이미지 분류 및 객체 감지 문제에서 기존의 컨볼루션 신경망(CNN)기반의 모델보다 Transformer를 활용한 Vision Transformer(ViT)[2]를 활용한 모델에서 더 많은 최첨단 성능을 달성하였으나, ViT기반의 모델들은 거대한 파라미터를 가지는 것과 동시에 성능을 올리기 위해서 수많은 데이터가 필요하다는 단점을 지닌다.

우리의 연구에서는 도로 교통 상황에서 문제가 될 수 있는 도로의 Pothole에 대해서 분류 및 감지를 할 수 있도록 Pothole 데이터를 사용하며, 해당 데이터의 수는 ViT기반의 모델이 학습을 하기에 적은 양의 데이터를 가진다. 따라서 우리는 ViT기반의 모델의 단점인 많은 파라미터 및 데이터가 필요하다는 단점을 극복하기 위한 모델 및 학습 방법에 대해서 제안한다.

1.2 연구내용 및 구성

최근 10년 동안 딥러닝에 발전에 따라서 많은 딥러닝 모델이 나왔으며, 기술이 발전함에 따라 GPU 기반의 컴퓨팅 자원을 바탕으로 매우 깊은 Layer와 많은 수의 파라미터를 가진 딥러닝 모델이 나오고 있지만, 컴퓨팅 리소스가 충분하지 않은 IoT 장비나 Edge Device에서는 이러한 딥러닝 모델을 적용하기에는 모델의 연산에 필요한 자원이 부족하다[1]. 따라서, 모델의 성능은 최대한 유지하면서 적은 연산량으로 추론이 가능하도록 모델의 복잡도를 줄이는 모델 압축(Model Compression) 기법이 필수불가결하다. 하지만 모델 압축 기법을 사용하면 모델의 파라미터가 감소하여 모델의 정확도가 낮아질 수 있는 문제가 발생할 수 있다. 따라서, 정확도 감소율, 모델 압축률 등의 우선순위를 고려하여 가장 효과적인 압축 기법을 사용하는 것이 중요하다.

대표적인 모델 압축 기법으로는 깊고 큰 모델의 손실(Loss) 값을 얕고 작은 모델에 전이하면서 작은 모델의 성능을 올리는 지식 증류(Knowledge Distillation) 기법[3], 모델 내에서 매우 작은 값을 가지는 가중치를 0으로 설정하여 연결을 끊는 가중치 가지치기(Weights Pruning) [4] 기법, 부동 소수점으로 표현되는 가중치를 특정한 비트수로 줄여 모델의 크기를 줄이는 양자화(Quantization) [4] 기법 등이 있다. 우리의 연구에서는 다양한 압축 기법에 따른 모델의 크기(MB) 및 한 이미지 당 처리 시간(sec/img), 정확도를 비교하여 각 압축 기법이 모델의 성능에 얼마나 영향을 미치는지 연구하며 성능을 수치화하여 압축 방법에 따른 장단점에 대해서 분석을 진행했다.

최근 컴퓨터 비전 영역에서 최첨단 성능을 보이고 있는 ViT모델을 활용하여 ViT모델의 단점인 학습에 많은 데이터 필요, 복잡한 네트워크 아키텍처의 구성 또는 CNN 모델의 도움을 받아서 성능을 올리는 단점을 가진다.

따라서, 본 논문에서는 다양한 딥러닝 모델의 압축기법의 비교 및 분석을 통해 선정된 효과적인 압축기법을 ViT모델에 적용하여 Pothole 데이터와 같은 적은 데이터로도 효과적인 학습을 할 수 있는 모델을 제안한다.

논문의 구성은 다음과 같다. 2장에서 모델 경량화 기술 및 딥러닝 모델 및 분산 학습에 대해서 소개하고, 3장에서 모델 경량화 기술에 대한 실험과 결과에 대한 분석한

다. 4장에서 우리의 연구에서 사용된 Pothole 데이터에 대해서 소개 및 데이터 전처리 방법과 데이터 증분 방법에 대해서 설명한다. 5장에서 우리가 제안하는 SDiT 모델의 네트워크 아키텍처 소개와 손실 함수의 비교 및 Hyperparameter 튜닝 방법에 대해서 설명한다. 6장에서 SDiT 실험 환경 소개 및 실험 결과 분석을 진행한다. 마지막으로 7장에서 결론 및 향후 연구로 마무리한다.

제 2 장 관련 연구

2.1 모델 경량화 기술

Cheng의 연구 [5] 에서 기존 Deep Neural Network 모델의 추론 작업 수행 시 계산 비용이 많이 들며, Infernece 수행 작업은 메모리와 같은 컴퓨팅 리소스가 적은 장치에서 대기 시간이 엄격한 애플리케이션 배치하는 데 방해가 됨을 명시하였다. 또한 Nan의 연구 [6] 에서 모바일 및 임베디드 장치의 급속한 발전에도 불구하고, 모바일 및 임베디드의 장치에서 직접 계산하는 비용이 많이 들기 때문에, 딥러닝 모델을 모바일 및 임베디드 장치에 맞추기 위해 여러 압축 기술에 대해서 제시한다.

본 절에서 우리는 딥러닝 모델을 경량화하기 위해 사용되는 모델 압축 기법인 지식 증류(Knowledge Distillation), 가중치 가지치기(Weight Pruning), 양자화(Quantization) 기술에 대해서 분석한다.

2.1.1 지식 증류

G.Hinton의 연구 [3]에서 처음 제안된 지식 증류(Knowledge Distillation) 기법은 크고 복잡한 Teacher 모델의 학습 결과를 작고 경량화된 Student 모델에 전달하면서, 비슷한 정확도로 학습하는 방법이다. 이 기법은 딥러닝에서 모델 압축과 경량화 관점에서 매우 중요한 역할을 한다. 기존의 딥러닝 모델은 많은 파라미터와 복잡한 구조로 인해 학습과 추론에 많은 계산 리소스와 메모리 공간을 요구하며, 이로 인해 실제 서비스나 장치에서 모델 배포 및 실행에 어려움이 있다. 이를 해결하기 위해, 지식 증류 기법은 Teacher 모델에서 Student 모델로 지식을 전달하여 작은 모델이 비슷한 예측 능력을 갖도록 만든다. 이 방법을 통해 작은 크기의 모델이 컴퓨팅 리소스를 효율적으로 사용하면서도 Teacher 모델과 유사한 성능을 달성할 수 있다.

본 항에서는 G.Hinton이 제안한 지식 증류 기법 [3]과 L. Zhang이 제안한 자기 지식 증류(Self-Knowledge Distillation) 기법 [7]에 대해 두 기법의 차이점을 설명한다.

(1) 지식 증류(Knowledge Distillation)

G.Hinton이 제안한 지식 증류 기법은 아래 그림 2-1와 같은 구조로 Teacher 모델의 출력값을 이용하여 Student 모델로 전이시키는데 사용된다.

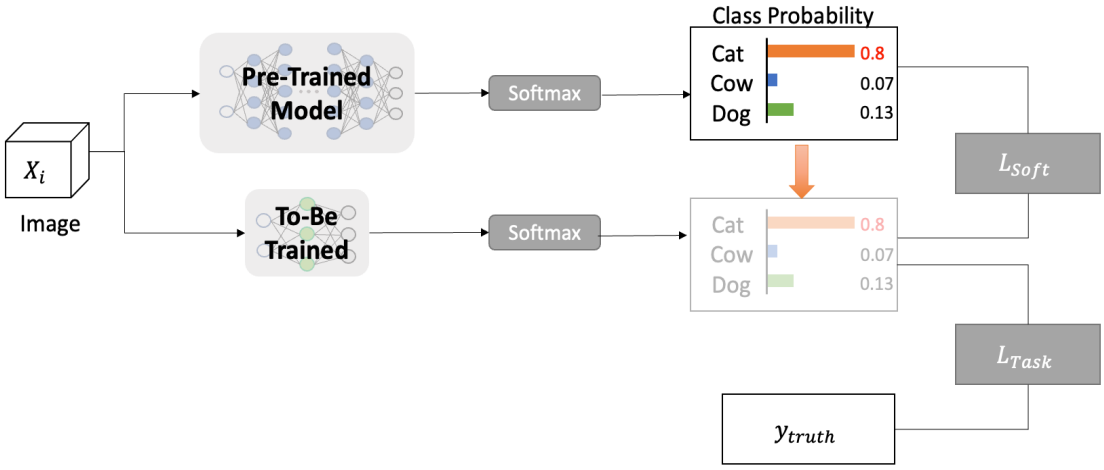


그림 2-1 G.Hinton의 지식 증류 구조

지식 증류에서는 기존의 Softmax 함수인 식 (1) 을 변형하여 스케일링 역할을 하는 τ(Temperature) 변수를 추가한 변형된 Softmax 함수인 식 (2) 를 사용합니다. 이 변형된 Softmax 함수의 출력은 분류하고자 하는 모든 클래스에 대한 확률 분포인 Soft Target으로 표현되며, 이를 활용하여 지식 증류에서 손실 함수 값을 계산합니다.

$$Softmax(Z_i) = \frac{\exp(z_i)}{\Sigma_j \exp(z_i)} \tag{1}$$

$$Softmax(Z_i, \tau) = \frac{\exp(z_i/\tau)}{\Sigma_j \exp(z_i/\tau)} \tag{2}$$

식 (2)를 통해 얻은 Teacher 모델과 Student 모델의 출력 값을 활용하여 Student 모델의 손실을 계산한다. 이 과정에서 Teacher 모델의 출력인 확률 분포와 Student 모델의 확률 분포 간의 차이를 줄이기 위해 식 (3) 과 같이 쿨백-라이블러 발산 (Kullback-Leibler Divergence, KLD)를 사용하여, Teacher 모델의 지식을 Student

모델로 전달하는 역할을 하는 손실 값(L_{Soft})을 구한다. 또한, 식 (4) 를 통해 Student 모델의 출력 값과 실제 정답인 y_{Truth}의 Cross-Entropy의 결과 값(L_{Task})을 구한다. 이렇게 얻은 L_{Task}와 L_{Soft}을 활용하여 전체 손실(L_{Total})을 식 (5) 를 통해 얻을 수 있다.

$$L_{Soft} = \sum_{x_i \in X} KLD(softmax(f_T(x_i), \tau),\ softmax(f_S(x_i), \tau)) \quad (3)$$

$$L_{Task} = CrossEntropy(softmax(f_S(x_i)), y_{truth}) \quad (4)$$

$$L_{Total} = \alpha L_{Task} + (1-\alpha) L_{Soft} \quad (5)$$

전체 손실(L_{Total})은 KLD와 Cross-Entropy 손실의 가중 합으로 구성된다. 가중치인 α는 KLD와 Cross-Entropy 손실 간의 상대적인 중요성을 조절하는 역할을 하며, 손실 함수를 최소화함으로써 Teacher 모델의 지식이 Student 모델로 전달되며, Student 모델은 Teacher 모델의 예측 능력에 근접하도록 학습된다. 이러한 지식 증류(Knowledge Distillation) 기법은 작은 모델(Student 모델)이 큰 모델(Teacher 모델)의 성능과 유사한 성능을 달성할 수 있는 모델 압축 및 경량화에 유용한 방법으로 활용된다.

(2) 자기 지식 증류(Self-Knowledge Distillation)

L. Zhang이 제안한 자기 지식 증류 기법 [7]은 기존의 지식 증류 기법은 Teacher 모델을 통해서 Student 모델의 성능을 높이는 방법에서 그림 2-2와 같은 동일한 모델 아키텍처를 공유하면서 성능을 높이는 효과를 보였다. 동일한 아키텍처에서 보조 분류기(Auxiliary Classifier)를 추가로 연결하여 각 보조 분류기에서 나오는 출력 값과 원래의 분류기에서 나오는 출력을 이용하여 자기 지식 증류를 수행한다.

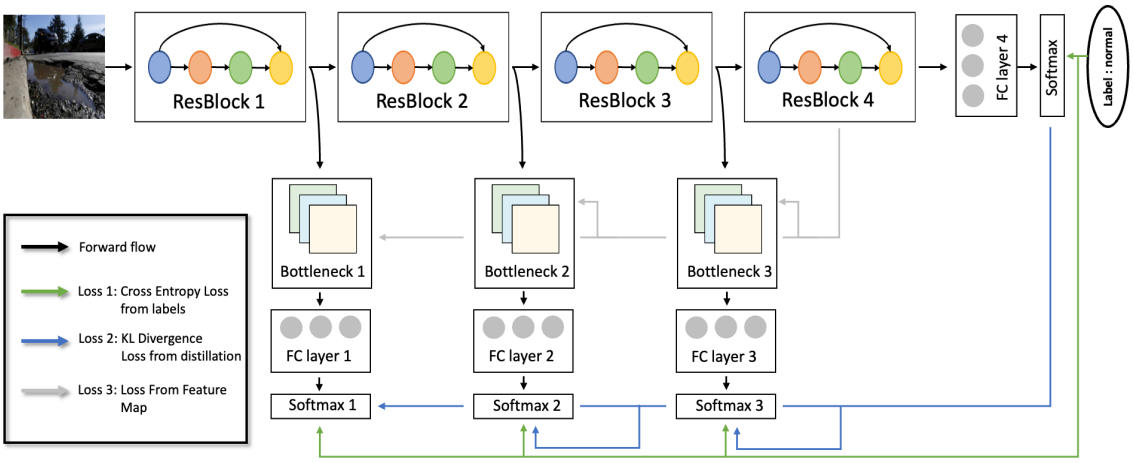


그림 2-2 자기 지식 증류 네트워크 아키텍처

L. Zhang이 제안한 자기 지식 증류 기법에서 손실 값은 다음과 같은 방법으로 구할 수 있다. 우선 그림 2-2에 나온 'Loss 1'은 식 (4)를 사용하여 실제 정답인 y_{Truth}와 각 보조 분류기에서 나온 예측 값 $y^i_{Predict}$ 의 Cross-Entropy 손실 값이며, 'Loss 2'는 식 (3)을 사용하여 원래 모델의 분류기에서 나온 결과 값($y^C_{Predict}$) 과 분류기에서 나온 각 결과 값($y^i_{Predict}$)을 사용하여 구할 수 있다. 하지만, 자기 증류 기법에서는 각 잔차 네트워크 블록(ResBlock) 다음에 추가된 병목 블록(Bottleneck Block)에서 특징(Feature)를 추출하여 원래 모델의 최종 잔차 네트워크 블록과 각 분류기에 있는 병목 블록간의 손실(L^i_{Feat})을 다음 식 (6) 과 같이 노름 공간(Norm Space) 연산을 사용하여 각 특징의 차이를 구한다. 각 3개의 손실 값을 통해서 전체 손실은 가중 합으로 구할 수 있으며 다음 식 (7) 과 같다.

$$L^i_{Feat} = \| F_i - F_C \|^2_2 \tag{6}$$

$$\begin{aligned} L_{Total} &= \sum_i^C loss_i \\ &= \sum_i^C ((1-\alpha) \bullet CrossEntropy(y^j_{Predict}, y_{Truth}) \\ &+ \alpha \bullet KLD(y^j_{Predict}, y^C_{Predict}) \\ &+ \lambda \bullet L^i_{Feat}) \end{aligned} \tag{7}$$

식 (7)에서 $i = C$ 인 마지막 상황에서는(원래 모델의 잔차 네트워크 블록일 때) 쿨백-라이블러 손실과 특징 추출 손실에 대해서는 0이 되어 Cross-Entropy연산의 손실 값만 합하여 전체 손실(L_{Total})을 구할 수 있게 된다. 기존의 지식 증류 기법과 다르게 자기 지식 증류 방법은 Teacher모델의 도움 없이 모델의 최종 출력 값과 각 모듈에 연결된 보조 분류기의 출력 값을 이용하여 네트워크 성능의 개선을 이루어 냈다.

2.1.2 가중치 가지치기

대부분의 딥러닝의 최종 모델에서 얼만큼의 파라미터가 최소한으로 필요한지 알 수 없다. 따라서 대부분의 딥러닝의 모델은 필요 이상의 많은 파라미터를 학습하게 된다. S. Han의 연구에서는 필요 이상의 많은 파라미터들이 추론에 동일한 영향을 주는 것이 아님을 보였다 [4]. 따라서 그림 2-3 과 같이 추론에 큰 영향을 미치지 않는 뉴런들 간의 연결을 제거하여 모델의 파라미터를 줄이는 모델 압축 기법인 가중치 가지치기가 필요하다.

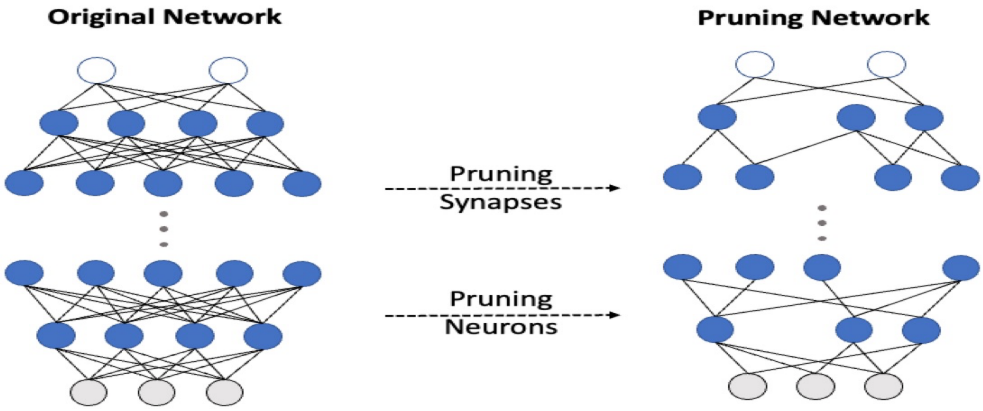


그림 2-3 가중치 가지치기 도식화

P. Molchanov의 연구 [8]에서는 가중치 가지치기를 통해 모델 크기를 크게 줄이고 희소성(Sparsity)을 도입하여 추론 과정에서의 메모리 사용량과 연산 비용을 상당히 감소시킬 수 있다는 것을 보여준다.

가중치 가지치기는 다음 그림 2-4 와 같은 단계로 이루어진다. 첫 번째 단계에서 사전 학습된 딥러닝 모델을 불러오거나 모델을 처음부터 학습을 한다. 두 번째 단계에서 각 가중치의 중요도를 계산하여 모든 가중치에서 우선순위를 정하며, 중요도가 낮은 가중치는 제거 대상이 된다. 세 번째 단계에서 사전 설정된 희소성 기준에 따라 중요하지 않은 가중치를 제거한다. 이 때, 모델 내의 불필요한 연결이 제거되고, 희소성이 도입되어 모델의 크기가 줄어들게 된다. 네 번째 단계에서 Fine Tuning을 통해 모델을 추가적인 학습을 수행하여 가중치 가지치기로 인한 성능 손실을 최소화시킬 수 있다. Fine Tuning 단계에서 모델의 크기, 정확도 성능, 추론 시간을 확인하여 가중치 가지치기의 효과를 평가하며, 평가를 기반으로 다시 가중치 가지치기를 진행할지 아니면 가중치를 고정을 통해 모델을 내보낼지 선택을 할 수 있다.

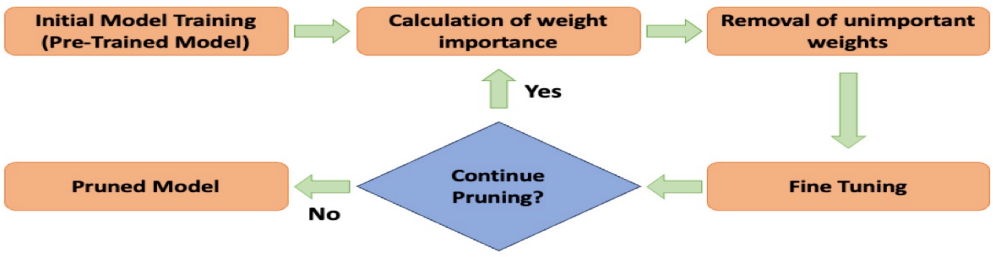


그림 2-4 딥러닝 모델의 가중치 가지치기 순서도

2.1.3 양자화

일반적인 딥러닝 모델은 float32 타입과 같이 32비트 또는 64비트의 고정 소수점(float) 형식을 사용하여 모델 파라미터와 연산을 표현 변수를 주로 사용한다. 그러나 이러한 고정 소수점 형식은 메모리 사용량과 연산 비용이 크기 때문에 IoT나 Edge Device와 같은 자원이 제한된 환경에서는 문제가 될 수 있다. 양자화는 이러한 문제를 해결하기 위해 모델 파라미터와 연산을 낮은 비트 수로 표현하는 방식이며, 아래

그림 2-5와 같이 가중치와 활성화 값을 정수(int) 형식 또는 작은 비트 수의 부동 소수점 형식으로 표현한다. 양자화 기법은 파라미터의 일부를 삭제하지 않고도 크기를 줄일 수 있다 [9].

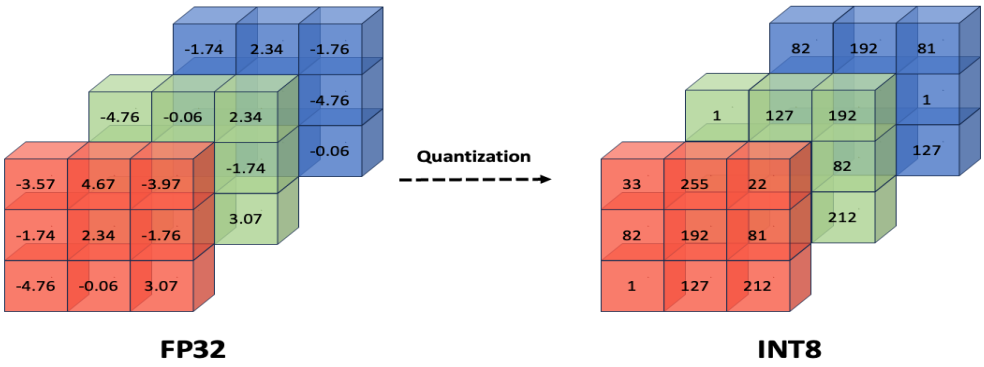


그림 2-5 고정 소수점 변수를 정수형 변수로 변환

양자화 기법에는 학습 후 양자화, 양자화 인식 학습이 있다. 학습 후 양자화는 모델의 성능을 거의 저하시키지 않으면서 CPU 및 하드웨어 가속기의 대기 시간을 개선하며 모델의 크기를 줄여주는 모델 압축 기법이다. 우리 연구에서는 딥러닝 모델의 양자화 기법 중 학습 후 양자화(Post-Training Quantization)와 양자화 인식 학습(Quantization-Aware Training)에 대한 연구를 진행했다.

학습 후 양자화 기법(Post-Training Quantization)은 딥러닝 모델의 가중치와 활성화 함수 값을 낮은 비트 수로 표현하는 과정을 말한다. 이를 통해 모델의 메모리 사용량과 저장 요구 사항을 줄이고, 추론 속도를 높일 수 있다. 학습 후 양자화 기법에서의 가중치 양자화는 모델의 학습 중에 사용된 가중치를 낮은 비트 수로 표현하는 방식으로, 고정점 양자화가 가장 널리 사용된다. 그리고, 활성화 양자화는 추론 과정에서 입력으로 들어가는 활성화 함수의 값을 낮은 비트 수로 표현하는 방법을 의미한다. 이때도 일반적으로 고정점 양자화 방식이 사용되며, 활성화 양자화는 각 층에서의 활성화 값을 정수형 변수로 변환하고, 고정된 스케일로 나누어 저장하여 연산 속도를 가속화할 수 있다.

양자화 인식 학습(Quantization-Aware Training)은 딥러닝 모델을 양자화 과정을

고려하여 모델을 학습하는 기법이며, 모델의 압축보다 양자화 된 모델의 성능 저하를 최소화하고 보다 정확한 양자화 된 모델을 생성하는 것을 목표로 한다. 양자화 인식 학습은 모델의 레이어, 손실 값, 가중치들을 양자화 시킨 후, 데이터의 입력을 통해 모델을 학습을 하는 과정을 거쳐 양자화 된 모델을 만들어 내는 것을 목표로 하며, 우리의 연구에서는 두 양자화 기법에 대해서 각각 진행을 한 결과와 두 양자화 기법을 동시에 적용을 했을 때의 결과를 같이 비교하며 분석한다.

2.2 딥러닝 모델

최근 몇 년 동안 많은 딥러닝 영역 많은 연구들이 진행 되었다. 본 절에서는 컴퓨터 비전에서 많이 사용되는 VGG, ResNet과 같은 전통적인 컨볼루션 신경망(CNN) 모델과 MobileNet, EfficientNet과 같은 IoT/Edge Device와 같은 소규모 장치를 위한 딥러닝 모델 그리고 마지막으로 최근 많은 연구가 진행이 되고 있는 Self-Attention 연산의 Transformer 모델을 활용한 Vision Transformer(ViT), DeiT 모델에 대해서 분석한다.

2.2.1 VGG

K. Simonyan의 연구 [10]에서 공개된 VGG 모델은 주로 이미지넷(ImageNet) 데이터 셋에서 대규모 이미지 분류 작업을 수행하는 데 사용되었다. 그림 2-6의 VGG16 모델의 아키텍처는 총 16개의 컨볼루션 레이어와 3개의 완전 연결(Fully connected) 레이어로 구성된다. 컨볼루션 레이어에서는 작은 필터와 풀링(Pooling) 레이어를 반복하여 사용하며, 이를 통해 신경망은 점진적으로 더 복잡한 패턴을 학습하며, 이미지의 공간적 특징을 다양한 수준에서 추출이 가능하다.

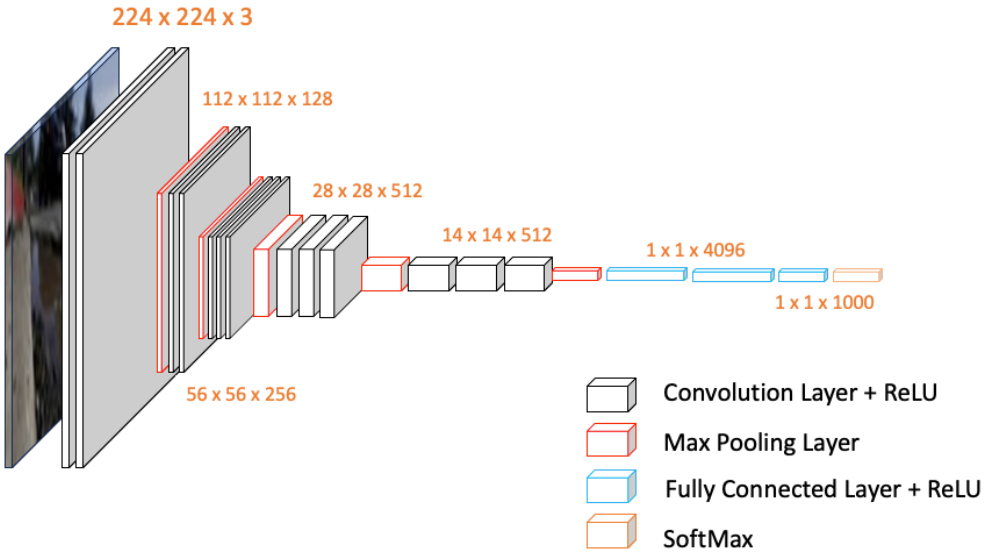


그림 2-6 VGG16 모델 아키텍처

VGG 모델의 주요 장점은 단순하면서도 효과적인 구조로 인해 이해하기 쉽고 구현하기 쉽다는 점과 작은 필터와 깊은 스택 형태의 구조를 통해 다양한 시각적 패턴을 학습할 수 있고, 일반화 능력이 뛰어나다는 점이다. 그러나, VGG 모델의 구조는 많은 파라미터를 가지고 있어서 모델의 크기가 크고 계산 비용이 많이 든다는 단점이 있지만 깊은 신경망 구조를 연구하는 데 큰 영향을 미쳤다.

2.2.2 ResNet

K. He의 연구에서 공개된 ResNet [11]은 "Residual Network"의 줄임말로, 깊은 신경망 모델 에서 발생하는 그래디언트 소실(Gradient Vanishing) 문제를 해결하기 위해 제안된 네트워크 구조다. ResNet은 Residual Block 이라는 블록(또는 모듈) 단위로 구성 되며, 블록의 구조는 다음 그림 2-7과 같다. Residual Block은 특징 중 첫 번째는 Skip Connection을 도입하여 그래디언트 소실 문제를 해결했다. Skip Connection은 입력 값을 출력 값에 더하는 연결을 의미하며, Skip Connection을 사용한 블록의 입력 값과 출력 값의 관계는 다음 식 (8) 과 같이 나타낼 수 있다.

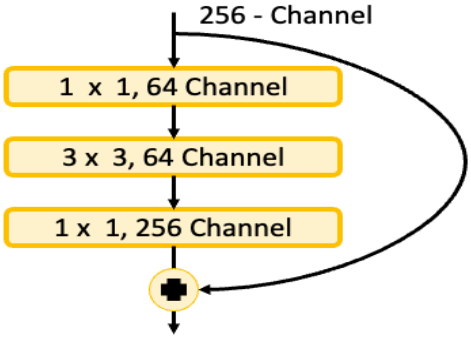


그림 2-7 Residual Block 구조

$$y_{Block} = F(x_{Input}) + x_{Input} \tag{8}$$

식 (8) 봤을 때, x_{Input}을 블록의 연산 함수(F)의 결과와 합하여 다음 블록의 입력으로 사용하게 된다. 이때 손실을 구하기 위한 역전파를 구할 때, x_{Input}이 따로 있는 구조로 인하여 그래디언트 소실 문제를 방지하게 된다.

Residual Block의 두 번째 특징으로는 병목(BottleNeck) 구조이다. ResNet의 병목 구조는 ResNet Block을 사용하여 깊은 네트워크를 구축할 때 파라미터의 수를 줄이고 계산 효율성을 향상시키는 역할을 한다. Block의 첫 번째 층인 '1x1 Convolution' 레이어는 입력의 채널 수를 줄이는 데 사용되는 컨볼루션 연산을 수행한다. 이를 통해 모델의 파라미터 수를 줄이고 연산을 감소시키는 효과를 얻을 수 있다. 두 번째 층인 '3x3 Convolution' 레이어는 '1x1 Convolution' 이후에는 채널 수를 늘리기 위한 연산을 수행하며, 이 단계에서 네트워크가 특징을 추출하고 다양한 패턴을 학습할 수 있다. 마지막 층인 '1x1 Convolution'에서 입력과 같은 채널을 만들어 주는 역할을 수행한다. Bottleneck 구조에서 입력과 출력의 채널의 수는 같지만 결과적으로 연산에 필요한 파라미터의 수는 줄어들게 되어 파라미터 수를 제어하고 연산량을 줄이는 역할을 수행한다.

위와 같은 특징을 가지는 Residual Block 내부에서 컨볼루션, 배치 정규화(Batch Normalization), 활성화 함수(Activation), 풀링(Pooling) 등의 다양한 레이어를 조합

하여 입력 값의 복잡한 특징을 추출할 수 있다. 또한, 파라미터 수를 줄이는 효과와 그래디언트 소실을 방지하는 구조로 네트워크의 깊이를 늘릴 수 있는 장점을 가진다.

2.2.3 MobileNet

MobileNet [12]은 모바일 기기 및 IoT/Edge Device와 같이 리소스가 제한된 환경에서 효율적으로 동작하는 CNN기반의 모델이다. MobileNet은 경량화된 네트워크 구조로, 작은 모델 크기와 낮은 계산 비용을 유지하면서도 높은 정확도를 목표로 설계되었다. Depthwise Separable Convolution과 Pointwise Convolution 두 가지 연산을 조합하여 효율적인 연산을 수행한다. Depthwise Seperable Convolution은 그림 2-8과 같이입력 채널별로 따로따로 필터를 적용하는 연산이며, 이는 입력 채널 간의 상호작용을 줄이고 계산 비용을 줄여 입력 채널마다 독립적으로 필터링을 수행하여 작은 모델 크기와 더 적은 계산 비용을 가질 수 있다. Pointwise Convolution은 1x1 convolution 연산으로, 입력 채널의 선형 결합을 통해 채널 수를 줄이거나 늘리는 역할을 하며, 채널 간의 관계를 학습하고, 네트워크의 표현력을 증가한다.

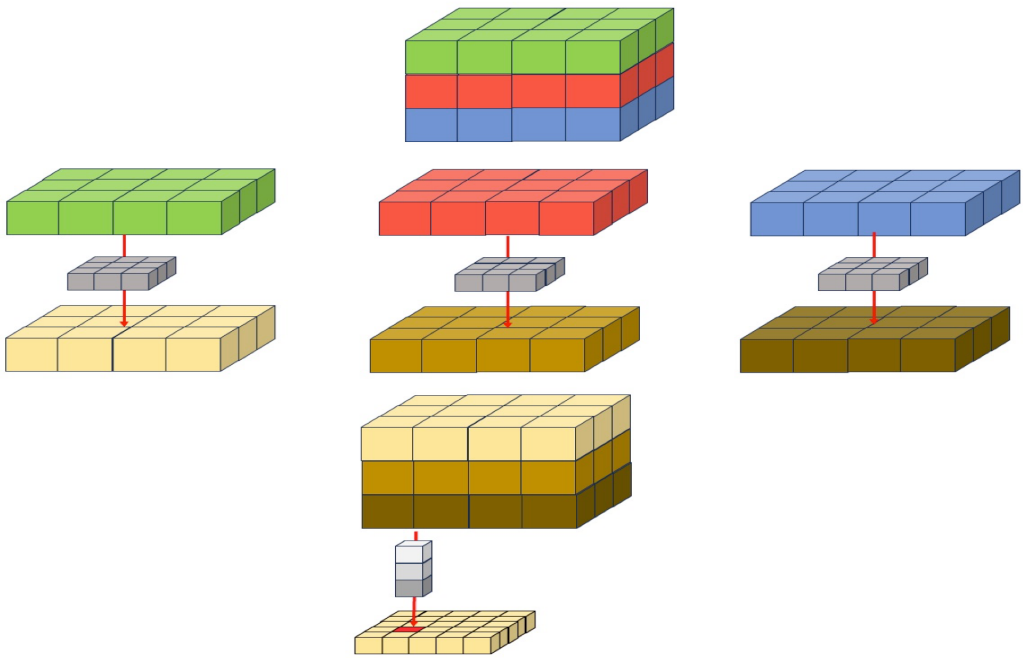


그림 2-8 Depthwise Separable Convolution과 Pointwise Convolution 구조

MobileNet은 Depthwise Separable Convolution을 여러 번 쌓아서 심층 네트워크를 구성한다. MobileNet은 작은 모델 크기와 높은 계산 효율성을 가지면서도 이미지 인식, 객체 검출 및 분할과 같은 컴퓨터 비전 작업에서 뛰어난 성능을 보인다. 또한, MobileNet은 모바일 기기에서의 배포와 실행에 특화되어 있으며, 실시간 응용 프로그램과 같이 실시간 처리가 필요한 시나리오에서 유용하게 사용된다.

2.2.4 EfficientNet

M. Tan에 의해 연구된 EfficientNet [13]은 모델의 규모를 조절하여 최적의 성능과 효율성을 달성하기 위한 네트워크 구조를 제시했다. 기존의 딥러닝 모델은 일반적으로 규모가 크고 복잡하여 많은 리소스를 요구했다. 하지만 EfficientNet은 계산 비용을 줄이고 모델의 크기를 최적화하여 효율적인 모델을 제시한다.

EfficientNet은 네트워크 구조의 규모를 조절하기 위해 다음과 같은 세 가지 주요한 요소를 사용했다. 첫 번째로 규모 조절 계수(Compound Scaling Coefficient)이다. 규모 조절 계수는 네트워크의 깊이, 너비, 해상도를 조절하는 역할하며, 이 계수를 통해 모델의 전체적인 규모를 조절할 수 있다. 규모 조절 계수를 증가시키면 모델의 규모가 커지며, 성능이 향상 된다. 두 번째로는 깊이 스케일링 (Depth Scaling)이다. 깊이 스케일링은 모델의 깊이를 조절하는 방법이며, 깊이를 조절함으로써 네트워크가 더 많은 계층을 가지게 되어 이미지의 더 복잡한 특징을 학습할 수 있다. 깊이 스케일링을 통해 모델의 표현력이 향상되고, 복잡한 시각적 패턴을 인식할 수 있게 된다. 마지막으로 너비 스케일링 (Width Scaling)이다. 너비 스케일링은 각 레이어의 채널 수를 조절하여 모델의 계산 비용을 조절 가능하다. 너비를 조절함으로써 모델의 파라미터 수가 줄어들고 계산 리소스를 절약할 수 있다. 너비 스케일링은 모델의 효율성을 높여 모바일 기기, IoT/Edge Device와 같은 제한된 환경에서의 추론을 가능하게 하는 역할을 한다.

EfficientNet은 이러한 특징은 Neural Architecture Search(NAS)라고 불리는 최적화 알고리즘을 사용하여 제한된 하드웨어 리소스에 최적화된 모델 구조를 찾을 수 있

다. NAS는 이미지 분류와 같은 컴퓨터 비전(Computer Vision) 작업에 대해서 최적의 구조를 탐색하기 위해 자동화된 방법을 사용하며, EfficientNet은 여러 모델 구조를 자동으로 탐색하고 평가하여 최적의 성능과 효율성을 갖는 구조를 선택한다.

제한된 리소스를 가진 하드에어를 위한 모델인 EfficientNet과 MobileNet의 차이점은 다음과 같다. 첫 번째로는 구조의 차이이며, MobileNet은 Depthwise Separable Convolution을 중심으로 한 경량화된 네트워크 구조를 갖지만, EfficientNet은 네트워크 규모를 자동으로 조절하는 알고리즘인 NAS를 사용하여 최적의 구조를 탐색한다. 두 번째로는 네트워크의 규모에서 차이점이 생긴다. MobileNet은 작은 모델 크기와 낮은 계산 비용을 갖는 경량화된 네트워크를 목표로 설계가 되었지만, EfficientNet은 모델의 규모를 늘리면서 더 높은 성능을 달성하는 것을 목표로 한다. 즉, MobilNet은 최대한 리소스를 줄이는 방향으로 설계가 되었지만,EfficientNet은 사용가능한 자원을 최대한 사용하기 위해 설계된 모델이다. 마지막 차이점으로는 성능 측면에서 EfficientNet이 MobileNet보다 높은 정확도를 보이는 경향이 있다는 점이다. EfficientNet은 네트워크 규모를 자동으로 조절하여 여러 규모의 네트워크를 생성할 수 있으며, 이는 더 많은 파라미터와 표현력을 갖는 모델을 만들 수 있게 한다.

2.2.5 Transformer and ViT

최근 자연어 처리(Natural Language Processing, NLP) 영역에서 기존의 순환 신경망(Recurrent Neural Network, RNN) 기반의 모델에서 Self-Attention 연산 기반의 Transformer를 사용한 모델이 최고 성능을 보였다. Attention 연산은 주어진 쿼리(Query)와 일련의 키(Keys), 값(Values) 쌍을 가지고 주어진 쿼리에 관련된 정보를 찾아내는 연산 메커니즘이다. Attention 연산은 키와 쿼리 사이의 유사도를 계산하고, 이 유사도를 사용하여 값에 가중치를 부여하여 출력을 계산한다. 이를 통해 Attention 기법은 기존의 RNN, LSTM과 달리 입력 시퀀스의 다른 위치들 간의 상호작용을 모델링하며, 이는 입력 시퀀스 내의 다른 위치들 간의 의존성을 효과적으로 학습할 수 있도록 도와준다.

Transformer는 아래 그림 2-9와 같은 Scaled Dot-Product Attention과 Multi-Head Attention을 사용하여 Transformer의 Encoder와 Decoder를 구성한다. Scaled Dot-Product Attention은 다음 식 (9) 와 같이 표현되며, 입력 시퀀스 내의 다른 위치간의 상호작용을 모델링 하는 역할을 수행한다. Multi-Head Attention은 Scaled Dot-Product Attention을 여러 번 적용하여 다양한 관점에서 정보를 추출하는 메커니즘으로 동작한다. Multi-Head Attention은 다음 식 (10) 와 같이 표현되며, 입력으로 주어진 쿼리, 키, 값 행렬들을 병렬로 독립적으로 여러 개의 Attention 연산에 적용하고, 그 결과를 결합하여 최종 출력을 생성한다. Multi-Head Attention은 모델의 표현력을 향상시키고, 다양한 관점에서 입력 시퀀스를 파악할 수 있도록 한다. 이를 통해 Transformer 모델은 복잡한 관계를 모델링하고 다양한 작업에 효과적으로 적용할 수 있다. 그림 2-9와 같이 Transformer는 인코더(Encoder)-디코더(Decoder) 구조를 가진 딥러닝 모델로, 기계 번역 및 자연어 처리와 같은 시퀀스 기반 작업에 주로 사용된다. 인코더는 입력 시퀀스를 임베딩하고 압축하는 역할을 하며, 디코더는 압축된 정보를 기반으로 출력 시퀀스를 생성한다.

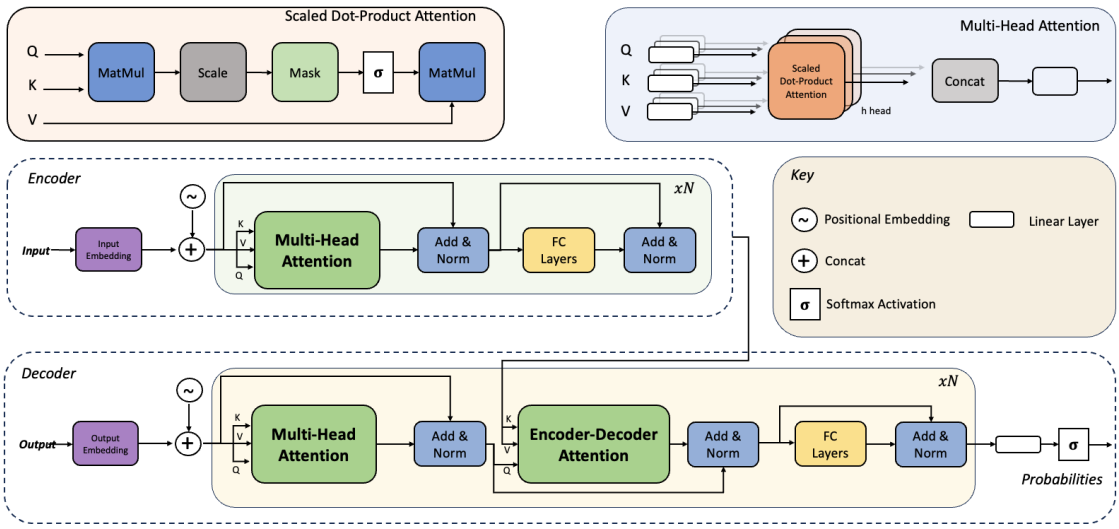


그림 2-9 Transformer 모델 구조

$$Attention(Q,K,V) = softmax(\frac{QK^T}{\sqrt{d_K}} \circ M)V \quad (9)$$

$$MultiHead(Q,K,V) = Concat(head_1, \cdots, head_h)W^O \quad (10)$$
$$where\ head_i = Attention(QW_i^Q,\ KW_i^K,\ VW_i^V)$$

자연어 처리에서 Transformer 기반의 모델들이 뛰어난 성능을 보여주며 이를 컴퓨터 비전 영역에도 적용을 하게 되면서 ViT 모델이 나오게 되었다. 컴퓨터 비전 영역에서도 최신성능을 보이고 있는 ViT(Vision Transformer)는 이미지를 처리하기 위해 그림 2-10과 같이 Transformer의 인코더만 사용하는 구조를 가진다. ViT는 이미지를 패치로 분할하여 Transformer에 입력하는 아이디어를 가지고 설계 되었다. 일반적인 컨볼루션 신경망(CNN)은 이미지를 지역적인 패치로 분석하는 방식을 사용되지만, ViT는 이미지를 패치로 분할하여 각 패치를 입력으로 사용하고, 패치들의 순서 정보를 Positional Embedding을 통해 전달하는 방식으로 Transformer에 적용했다.

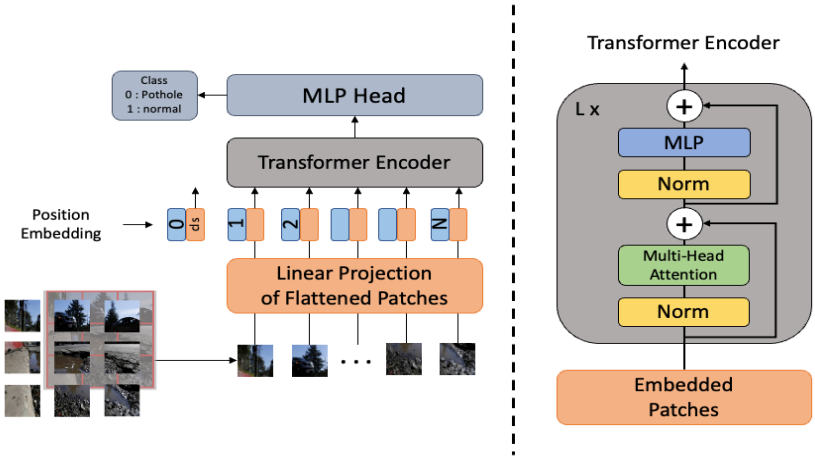


그림 2-10 Vision Transformer(ViT) 모델 구조

이미지를 패치로 분할하고 Transformer의 인코더에 입력으로 제공함으로써 ViT는 이미지 내의 지역 정보를 잃지 않고 전역적인 관계를 학습할 수 있다. 이는 CNN과는 다른 방식으로 이미지를 처리하면서, 픽셀 수준의 공간적인 변환을 적용하지 않고 전체적인 이미지에 대한 컨텍스트를 파악할 수 있는 장점을 가진다. 또한, 인코더만 사

용하는 이유는 이미지 분류 작업에서는 이미지를 특징 벡터로 변환한 후, 이를 분류기에 입력으로 사용하기 때문에, ViT에서는 인코더 부분만 사용하여 이미지의 특성을 추출하고 이미지 분류 작업에 활용할 수 있다. ViT의 인코더에서 이미지 내의 패턴을 인코딩하여 이미지의 정보를 추출한다. ViT는 이미지 분류 외에도 객체 검출, 세그멘테이션 등의 다른 컴퓨터 비전 작업에도 적용될 수 있다.

2.2.6 DeiT

기존의 ViT가 JFT-300M이라는 Google의 공개되지 않은 데이터를 사용하여 학습을 하면서 높은 성능을 보였다. H. Touvron의 연구진은 이러한 공개되지 않은 매우 많은 양의 데이터를 가지고 학습을 하며 높은 성능을 보인 ViT를 상대적으로 적은 데이터만을 사용해서 효율적인 학습 성능을 보인 DeiT(Data-efficient image Transformers)에 대해서 연구했다 [14]. DeiT는 다음 그림 2-11과 같은 구조를 지니며, 기존의 ViT의 모델 구조를 크게 변형하지 않고 사용했다.

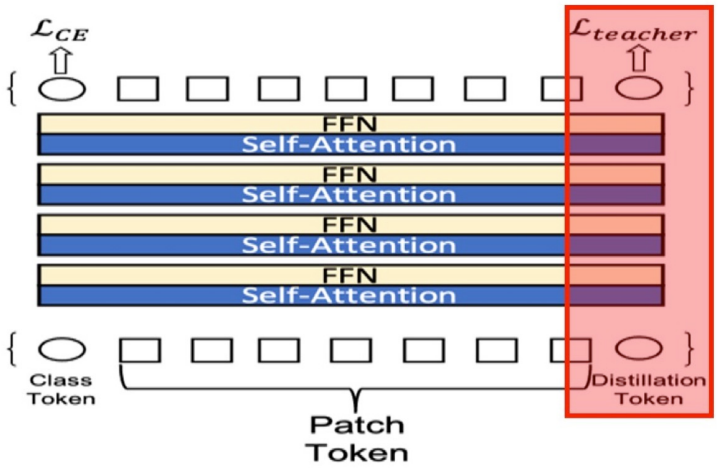


그림 2-11 DeiT(Data-efficient image Transformer) 모델 구조

DeiT는 Vision Transformer (ViT) 모델의 한계를 극복하기 위해 지식 증류 기법과 데이터 증강 기법을 효과적으로 결합하여 데이터 효율성과 성능을 개선한 이미지 인

식 모델이다. ViT는 대규모 데이터에서 사전 학습된 모델을 이용하여 높은 성능을 달성할 수 있지만, 작은 규모의 데이터에서는 한계를 보였다. DeiT는 이러한 한계를 극복하기 위해 지식 증류 기법을 도입했으며, 이를 위해서 그림 2-11과 같이 Distillation Token을 입력에 추가하여 Teacher 모델의 결과를 DeiT의 학습에 활용한다. 이를 통해 Teacher 모델이 학습한 지식을 DeiT 모델로 전달하며, 높은 성능을 발휘할 수 있도록 했다.

또한, DeiT는 데이터 증강 기법을 적극적으로 활용하여 작은 규모의 데이터에서도 성능을 향상시켰다. 데이터 증강은 입력 이미지에 다양한 변형을 적용하여 학습 데이터를 확장하는 과정을 말하며, DeiT는 ImageNet과 같은 대규모 데이터셋서 미리 학습된 가중치를 이용해 전이 학습(Transfer Learning)에 활용한다. 이를 통해 DeiT는 작은 데이터에서도 높은 성능을 달성할 수 있으며, 데이터의 다양성과 일반화 능력을 향상시켰다.

제 3 장 모델 경량화 기술 분석

이번 장에서는 앞서 소개한 모델 경량화 기술을 활용하여 세 가지 주요 기준인 정확도, 모델 크기, 처리 속도에 대한 비교 및 평가한다. 이를 통해 다양한 압축 방법들의 장단점과 성능 차이에 대해서 분석 할 수 있다. 우선, 정확도 측면에서 각 압축 방법들의 성능을 수치로 비교하며, 실험 결과를 통해 각 압축 방법이 얼마나 원본 모델의 정확도를 보존하는지 확인한다. 이를 통해 압축 방법의 정확도에 대한 신뢰성을 파악할 수 있다. 또한, 모델 크기 측면에서는 각 압축 방법들이 모델의 파라미터 수나 메모리 요구량을 어떻게 줄이는지 확인한다. 각 압축 방법이 얼마나 효과적으로 모델 크기를 줄이는지를 비교한다. 마지막으로, 처리 속도 측면에서는 각 압축 방법이 추론 시간을 어떻게 영향을 주는지 평가하며, 각 압축 방법이 얼마나 빠른 처리 속도를 제공하는지를 분석한다.

3.1 데이터 소개

모델 경량화 기술 분석을 위해 사용한 데이터는 다음 표 3-1과 같다. ImageNet은 컴퓨터 비전 연구에 있어 대표적인 대규모 데이터셋이다. ILSVRC(ImageNet Large Scale Visual Recognition Challenge)로 잘 알려진 국제 대회에서 사용되는 데이터로 유명하며, 본 연구에서 사용되는 사전 학습된 모델에 사용된 데이터셋이 ILSVRC ImageNet 데이터이다.

표 3-1 ImageNet, CIFAR-10 데이터셋 정보

	ImageNet(ILSVRC)	CIFAR-10
Num of Classes	1000	10
Num of data	1,431,167	60,000
Size of Image	Variety	32x32
Dataset size	about 155 GB	163 MB

CIFAR-10은 10개의 클래스로 구분된 사물 사진을 모은 dataset으로, 주로 아이디어에 대한 검증이나, ImageNet을 학습시키기 버거운 경우 사용한다. 본 연구에서는 Edge Device에서 추론 과정에서 사용할 가벼운 데이터가 필요하여 해당 데이터를 사용했다.

본 장의 실험에서 사용하는 모델은 전이학습을 사용한다. 특징을 추출하는 Convolution Layer는 ImageNet ILSVRC 데이터로 충분히 학습이 되어있으므로 학습 과정에서 가중치를 고정시키고, 분류기인 Fully Connected Layer는 CIFAR-10 데이터셋을 사용해 10개의 Class 분류가 가능하도록 재학습시켰다.

3.2 실험 환경

본 연구에서는 엣지 장치로 NVIDIA Jetson Nano를 사용하였다. 추론 시간 측정에 앞서, 비교 분석에 사용될 모든 모델들을 GPU 서버에서 학습시킨다. 이후 GPU 서버와 NVIDIA Jetson Nano에서 추론을 진행했다.

3.2.1 GPU 서버

GPU 서버의 스펙은 표 3-2와 같으며, GPU 서버 환경은 NVIDIA RTX 2080Ti GPU 8개가 장착되어 있으나 단일 GPU 환경에서 모델의 학습과 추론 시간 측정을 위해, GPU 1개의 자원을 쓸 수 있는 도커 컨테이너(Docker Container)에서 모델의 학습과 추론 시간 측정을 진행하였다. 해당 컨테이너는 Docker 20.10.2 버전이며, Tensorflow 2.4.1 버전이 사용 가능한 컨테이너를 구축했다.

표 3-2 GPU 서버 스펙 시트

GPU Server	
OS	CentOS 7.9
CPU	Intel(R) Xeon(R) Silver 4210 CPU *2
GPU	RTX 2080Ti * 8
RAM	125 GB
Docker	20.10.2
Tensorflow	2.4.1
Tensorflow -model -optimization	0.5.0

3.2.2 Edge Device: NVIDIA Jetson Nano

NVIDIA Jetson Nano의 스펙은 다음 표 3-3과 같으며, GPU 서버의 Tensorflow 버전은 2.4.1이다. 해당 버전에서 학습된 모델은, 다른 실험 환경에서 추론 진행 시 Tensorflow 2.3 이상의 버전이어야만 호환성 문제가 발생하지 않는다. 따라서, GPU 서버로 학습된 모델의 추론를 NVIDIA Jetson Nano로 측정하기 위해, NVIDIA Jetson Nano에 Tensorflow 2.3.1 버전을 사용할 수 있는 4.5.1 버전의 jetpack SDK가 포함된 Ubuntu 18.04.5 LTS 를 설치하였다.

표 3-3 NVIDIA Jetson Nano 스펙 시트

NVIDIA Jetson Nano	
OS	Ubuntu 18.04.5 LTS
CPU	Quad-core ARM A57 @ 1.43 GHz
GPU	128-core Maxwell
RAM	4 GB 64-bit LPDDR
Tensorflow	2.3.1 + nv 20.12

3.3 실험 결과 및 분석

3.3.1 실험 결과

본 연구에서는 데이터셋과 모델에 따라 ImageNet으로 사전 학습된 VGG19, ResNet50 모델을 CIFAR-10 데이터로 재학습 시킨 뒤, 베이스 모델과 3가지 모델 압축 기법인 지식 증류, 가중치 가지치기, 양자화의 평가 지표(정확도, 이미지 당 처리시간, 모델 크기)를 측정하였다. 테스트 이미지로는 CIFAR-10 테스트 데이터를 사용하였으며, 10000개의 모든 이미지를 가지고 엣지 장치: NVIDIA Jetson Nano 환경에서 테스트를 진행하였을 때, 심한 발열과 불규칙한 freezing이 발생하였다. 이를 고려하여 모든 실험 환경에서 100개의 CIFAR-10 test data에 대한 테스트를 진행하였다.

본 연구에서는 CIFAR-10 데이터셋과 사전 학습된 VGG19 및 ResNet50 모델을 활용하여 세 가지 모델 압축 기법인 지식 증류, 가중치 가지치기, 양자화를 평가했다. 실험에서는 압축 기법들의 성능을 비교하기 위해 정확도, 이미지 당 처리 시간, 모델 크기라는 세 가지 평가 지표를 사용했다.

실험은 엣지 장치인 NVIDIA Jetson Nano 환경에서 진행되었으나, 이 과정에서 심한 발열과 불규칙한 프리징(Freezing) 현상이 발생했다. 따라서 발열과 프리징(Freezing) 현상을 최소화하기 위해 모든 실험 환경에서는 100개의 CIFAR-10 테스트

데이터에 대한 테스트를 진행했다.

평가 지표인 정확도는 모델의 분류 성능을 나타내며, 이미지 당 처리 시간은 모델이 한 장의 이미지를 처리하는 데 걸리는 시간을 의미하며, 빠를수록 더 효율적인 모델이다. 모델 크기는 모델의 파라미터 수나 메모리 요구량을 나타내며, 작을수록 모델의 용량을 줄이는 데 효과적이다. 이러한 평가 지표들을 통해 각 압축 기법들의 성능을 비교하고, 각 기법의 장단점을 분석한다.

표 3-4와 3-5는 GPU 서버에서 VGG19와 ResNet50의 압축 방법들의 각 평가 지표인 정확도(acc) 이미지 당 처리시간(sec/img), 모델 파일 크기(MB)를 정리한 것이다. VG는 VGG19 모델과 GPU 서버 환경을 의미하며, RG는 ResNet50 모델과 GPU 서버 환경을 의미한다. KD는 지식 증류, KD 뒤에 붙은 모델은 student 모델을 의미하며, Q-aware는 양자화 인식 훈련, Q-Conv는 베이스 모델 훈련 후 양자화, Q-Dual은 양자화 인식 훈련 적용 모델 훈련 후 양자화를 의미한다.

표 3-4 VGG19의 GPU서버에서의 실험 결과 표

VGG19 GPU Server (VG)							
	Base	KD-CNN	KD-LeNet	Pruning	Q-aware	Q-Conv	Q-Dual
VG-Acc	82.99	10.00	41.99	61.00	70.99	84.00	75.00
Base 대비 VG-Acc 변화량	0	-0.88	-0.49	-0.26	-0.14	+0.01	-0.10
VG-time (sec/img)	0.219	0.037	0.038	0.049	0.046	0.484	0.486
Base 대비 VG-time 변화량	0	-0.83	-0.83	-0.78	-0.79	+1.21	+1.22
VG-Size (MB)	988.8	3.9	21.4	532.6	532.6	266.3	266.3
Base 대비 VG-Size 변화량	0	-1.00	-0.98	-0.46	-0.46	-0.73	-0.73

표 3-5 ResNet50의 GPU서버에서의 실험 결과 표

ResNet50 GPU Server (RG)							
	Base	KD-CNN	KD-LeNet	Pruning	Q-aware	Q-Conv	Q-Dual
RG-Acc	76.99	10.99	43.00	61.00	77.99	82.00	86.00
Base 대비 RG-Acc 변화량	0	-0.86	-0.44	-0.21	-0.01	+0.07	+0.12
RG-time (sec/img)	0.059	0.037	0.363	0.102	0.059	0.128	0.126
Base 대비 RG-time 변화량	0	-0.37	+5.15	+0.73	0	+1.17	+1.14
RG-Size (MB)	90.4	3.9	21.4	90.4	90.4	44.9	44.9
Base 대비 RG-Size 변화량	0	-0.96	-0.76	0	0	-0.50	-0.50

각 지표별 세부 비교는 다음 그림 3-1, 3-2, 3-3에서 자세히 설명한다. 그림 3-1의 그래프에 따르면 지식 증류(KD) 기법을 사용 할 때, 다른 압축 기법에 비해 정확도(Acc)의 손실률이 상당함을 확인하였다. VGG19 모델과 GPU 서버 환경(VG)의 경우 다른 모델 압축 기법에 비해 지식 증류의 VG-Acc가 약 19~74% 낮으며, 지식 증류를 사용한 ResNet50 모델과 GPU 서버 환경(RG)의 RG-Acc는 다른 모델 압축 기법에 비해 약 18~75% 낮았다.

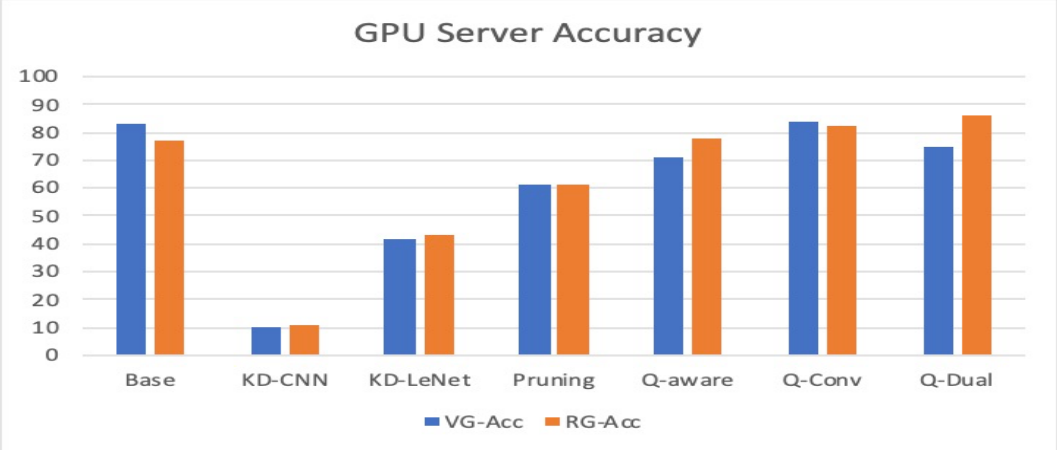


그림 3-1 GPU Server에서의 정확도 비교

그림 3-2의 그래프에 따르면, 지식 증류를 사용한 경우 KD-LeNet의 VG-time(sec/img) 감소율은 약 96.16%, KD-CNN의 VG-time(sec/img) 감소율은 약 82.44% KD-CNN의 RG-time(sec/img) 감소율은 약 38.55%, KD-LeNet의 RG-time 감소율은 약 38.88%로, 이미지 당 처리시간이 다른 기법에 비해 크게 감소하였다. 하지만, 학습 후 양자화 기법을 사용했을 때는 Q-Conv의 VG-time증가율은 약 121.20%, Q-Dual의 VG-time증가율은 약 122.02%, Q-Conv의 RG-time증가율은 약 116.16%, Q-Dual의 RG-time증가율은 약 112.45% 만큼 오히려 베이스 모델 보다 처리시간이 더 길어졌다.

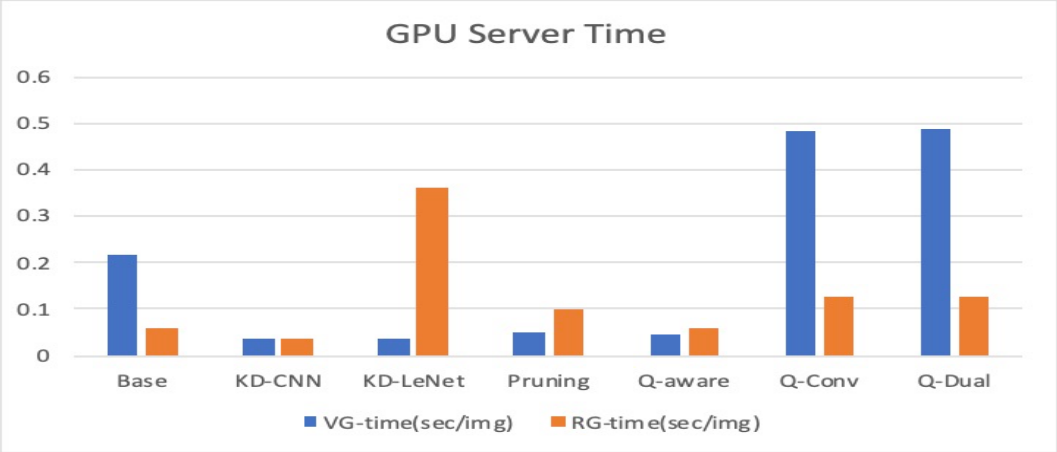


그림 3-2 GPU Server에서의 이미지 당 처리시간 비교

그림 3-3의 그래프에 따르면, 모든 모델 압축 기법 적용 시 베이스 모델 기준으로 모델의 파일 크기가 줄어들었다. 특히 지식 증류 기법을 사용한 경우 높은 압축률을 보였으며, 그 중에서도 CNN 모델로 압축했을 때 가장 큰 압축률을 보였다. VG에서는 지식 증류 기법을 사용한 KD-CNN의 압축률이 99.60%로 가장 높았으며 KD-LeNet은 97.83%로 그 뒤를 이었다. RG에서도 지식 증류 기법을 사용한 KD-CNN의 압축률이 95.68%로 가장 높았으며 KD-LeNet은 76.32%로 2번째로 압축률이 높았다.

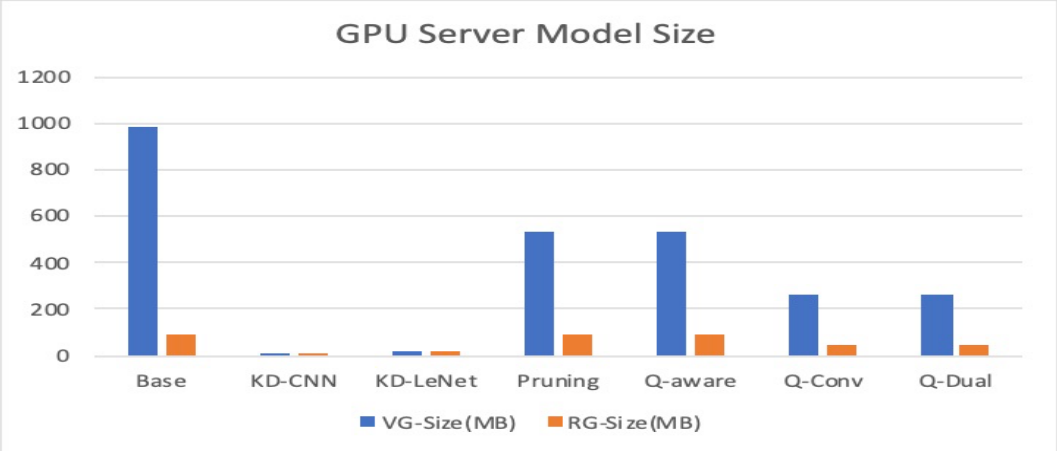


그림 3-3 GPU Server에서의 모델 파일사이즈 비교

표 3-6와 3-7는 Edge Device인 NVIDIA Jetson Nano에서 VGG19와 ResNet50의 압축 기법들의 평가 지표인 정확도(acc), 이미지 당 처리시간(sec/img), 모델 파일 크기(MB)를 정리한 것이다. VN는 VGG19 모델과 NVIDIA Jetson Nano 환경을 의미하며, RN는 ResNet50 모델과 NVIDIA Jetson Nano환경을 의미한다. KD는 지식 증류, KD 뒤에 붙은 모델은 Student 모델을 의미하며, Q-aware는 양자화 인식 훈련, Q-Conv는 베이스 모델 훈련 후 양자화, Q-Dual은 양자화 인식 훈련 적용 모델 훈련 후 양자화를 의미한다.

표 3-6 VGG19의 Edge Device에서의 실험 결과 표

VGG19 NVIDIA Jetson Nano (VN)							
	Base	KD-CNN	KD-LeNet	Pruning	Q-aware	Q-Conv	Q-Dual
VN-Acc	83.99	10.00	38.99	62.00	68.00	82.00	77.00
base 대비 VN-Acc 변화량	0	-0.88	-0.54	-0.26	-0.19	-0.02	-0.08
VN-time (sec/img)	1.073	0.188	0.187	0.969	1.134	5.281	5.340
base 대비 VN-time 변화량	0	-0.82	-0.83	-0.10	+0.06	+3.92	+3.98
VN-Size (MB)	988.8	3.9	21.4	532.6	532.6	266.3	266.3
base 대비 VN-Size 변화량	0	-1.00	-0.98	-0.46	-0.46	-0.73	-0.73

표 3-7 ResNet50의 Edge Device에서의 실험 결과 표

RseNet50 NVIDIA Jetson Nano (RN)							
	Base	KD-CNN	KD-LeNet	Pruning	Q-aware	Q-Conv	Q-Dual
RN-Acc	81.99	18.48	36.00	64.99	85.00	82.00	77.00
base 대비 RN-Acc 변화량	0	-0.77	-0.56	-0.21	+0.04	0	-0.06
RN-time (sec/img)	0.634	0.120	0.205	0.665	0.697	1.073	1.065
base 대비 RN-time 변화량	0	-0.81	-0.68	+0.05	+0.10	+0.69	+0.68
RN-Size (MB)	90.4	3.9	21.4	90.4	90.4	44.9	44.9
base 대비 RN-Size 변화량	0	-0.96	-0.76	0	0	-0.50	-0.50

그림 3-4의 그래프에 따르면 지식 증류(KD) 기법 사용 시, 다른 압축 기법에 비해 정확도(Acc)의 손실율이 상당함을 확인하였다. VGG19 모델과 NVIDIA Jetson Nano 환경(VN)의 경우 다른 모델 압축 기법에 비해 KD의 VN-Acc가 약 23%~74% 낮으며, KD를 사용한 ResNet50 모델과 NVIDIA Jetson Nano 환경(VN)의 RN-Acc는 다른 모델 압축 기법에 비해 약 29%~67% 낮았다.

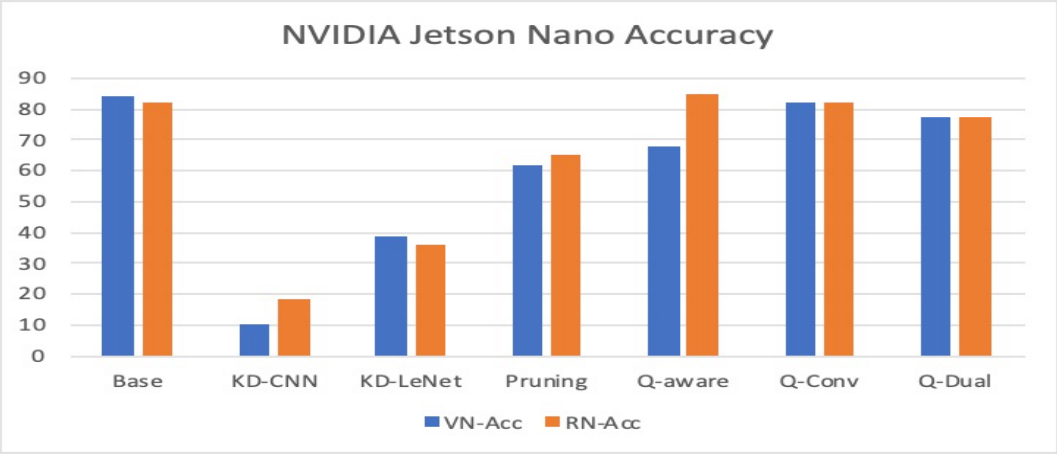


그림 3-4 Edge Device에서의 정확도 비교

그림 3-5의 그래프에 따르면, 지식 증류를 사용한 경우 KD-CNN의 VN-time(sec/img)의 감소율이 약 82.49% KD-LeNet의 VN-time 감소율 약 82.54%, KD-CNN의 RN-time(sec/img) 감소율은 약 81.09%, KD-LeNet의 RN-time감소율은 약 67.64%로, 이미지 당 처리시간이 다른 기법에 비해 크게 감소하였으나, 학습 후 양자화 기법을 사용했을 때는 Q-Conv의 VN-time증가율은 약 392%, Q-Dual의 VN-time증가율은 약 397%, Q-Conv의 RN-time증가율은 약 69%, Q-Dual의 RN-time증가율은 약 67.98% 으로, 오히려 Base Model보다 처리시간이 더 증가하였다.

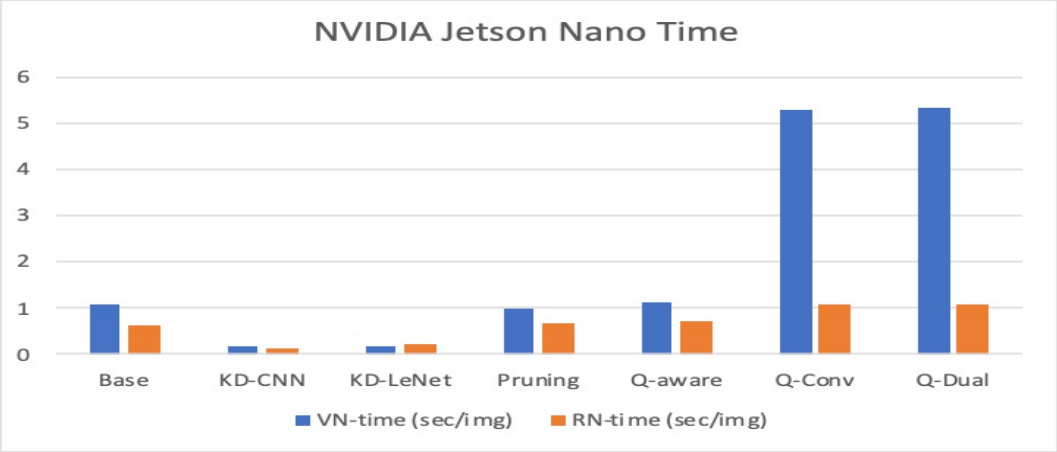


그림 3-5 Edge Device에서의 이미지 당 처리시간 비교

그림 3-6의 그래프에 따르면, 모든 모델 압축 기법 적용 시 베이스 모델 기준으로 모델의 파일 크기가 줄어들었다. 특히 지식 증류 기법을 사용한 경우 높은 압축률을 보였으며, 그중에서도 CNN 모델로 압축했을 때 가장 큰 압축률을 보였다. VG에서는 지식 증류 기법을 사용한 KD-CNN의 압축률이 99.60%로 가장 높았으며 KD-LeNet은 97.83%로 그 뒤를 이었다. RG에서도 지식 증류 기법을 사용한 KD-CNN의 압축률이 95.68%로 가장 높았으며 KD-LeNet은 76.32%로 2번째로 압축률이 높았다.

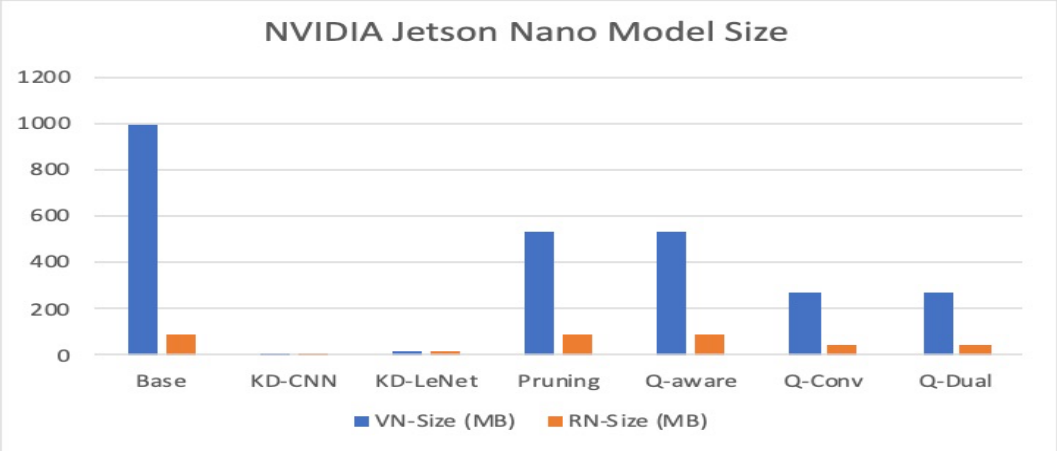


그림 3-6 Edge Device에서의 모델 파일사이즈 비교

3.3.2 결과 분석

실험 데이터 기반으로 각 모델 압축 기법의 장단점을 분석해보았다. 지식 증류 기법의 경우, 표 3-4, 표 3-5에 의하면 Teacher 모델로 사용한 베이스 모델인 VGG19, ResNet50의 모델 크기와는 상관없이 손실 값을 전이시킬 Student 모델인 CNN과 LeNet에 따라 모델의 파일 크기 및 처리 시간이 결정된다는 것을 확인되었지만, Student 모델의 학습 횟수가 적을 경우 낮은 정확도를 보인다는 것을 확인하였다.

가중치 가지치기의 경우, Tensorflow Model Optimization Toolkit의 공식 문서에 따르면[15] 설정한 희소성(Sparsity)값에 따라서 모델의 파일 크기와 정확도가 결정이 된다는 장점을 가진다. 하지만 너무 많은 가중치를 제거하게 될 때, 베이스 모델에 비해 정확도가 크게 떨어질 수 있다는 단점을 가진다. 본 연구에서는 희소성 값을 80으로 지정을 하여 전체 가중치의 20%만 남을 수 있도록 하였지만, 베이스 모델에 비해 정확도가 많이 떨어진 것을 학인했다. 실험 결과를 참고하여 가중치 가지치기를 적용을 할 때는 희소성 값을 조절하며 적절한 값을 찾는 것이 중요할 것이라 예상된다.

양자화 기법의 경우 높은 모델 파일사이즈 압축률과 정확도를 보였으나, 이미지 당 처리시간이 다른 모델 압축 기법에 비해 확연히 길어졌으며, 양자화 인식 훈련을 거치지 않고 훈련 후 양자화만 적용하였을 때, 충분한 정확도가 나온다면 양자화 인식

훈련은 오히려 정확도의 감소를 야기할 수 있다는 단점을 가진다.

본 실험의 경우 어떤 모델 압축 기법이 가장 우수한가 보다는, 압축 기법이 특정 상황에 따라 효과적인 기법을 가려내는 데 초점을 맞췄다. 따라서 평가 지표의 가중치를 각 (정확도 가중치), (처리 시간 가중치), (모델 크기)로 두고, 정확도, 처리 시간, 모델 크기의 3가지 측정치를 기준으로 동일한 정도의 중요도로 반영되도록 Min-Max 정규화를 통해 스케일링해준 뒤, 본 연구에서 새로 제안하는 식 (11)과 같이 베이스 모델과 압축 모델의 각 차이와 가중치를 수치화 하였다.

$$\begin{aligned} Score \;=\; & \alpha\,(ConvertModelAcc \;-\; -BaseModelAcc) \\ & +\; \beta\,(BaseModelTime \;-\; ConvertModelTime) \\ & +\; \gamma\,(BaseModelSize \;-\; ConvertModelSize) \end{aligned} \tag{11}$$

다음으로, 특정 평가 지표가 중요해지는 Case를 가정하여 표 3-8로 정리하였다. Case 1은 전부 동일한 가중치일 때이고, Case 2는 α(정확도)가 중요할 때, Case 3는 β(처리시간)가 중요할 때, Case 4는 γ(모델 크기)가 중요할 때이다. 각 Case마다 α, β, γ 의 값을 변화시켜 식 (11)을 계산한 후 표 3-9, 표 3-10에 정리하였다.

표 3-8 Case에 따른 가중치

	α	β	γ
Case 1	1	1	1
Case 2	2	1	1
Case 3	1	2	1
Case 4	1	1	2

표 3-9 GPU Server에서의 점수

VGG19 GPU Server (VG)							
	Base	KD-CNN	KD-LeNet	Pruning	Q-aware	Q-Conv	Q-Dual
Case 1	0	0.4194	**0.8297**	0.5451	0.6850	0.1570	0.0313
Case 2	0	-0.5670	0.2756	0.2479	**0.5228**	0.1706	-0.0766
Case 3	0	0.8251	**1.2312**	0.9241	1.0689	-0.4333	-0.5629
Case 4	0	1.4194	**1.8119**	1.0083	1.1482	0.8906	0.7649
ResNet50 GPU Server (RG)							
	Base	KD-CNN	KD-LeNet	Pruning	Q-aware	Q-Conv	Q-Dual
Case 1	0	0.3688	**0.5954**	-0.6714	0.0166	-0.1564	-0.0792
Case 2	0	-0.5111	**0.1422**	-0.8845	0.0299	-0.0896	0.0409
Case 3	0	0.6174	**0.8462**	-1.1296	0.0198	-0.9056	-0.8045
Case 4	0	1.3688	**1.3931**	-0.6714	0.0166	0.3696	0.4468

표 3-10 Edge Device에서의 점수

VGG19 NVIDIA Jetson Nano (VN)							
	Base	KD-CNN	KD-LeNet	Pruning	Q-aware	Q-Conv	Q-Dual
Case 1	0	0.1717	**0.5460**	0.1862	0.2352	-0.1099	-0.1890
Case 2	0	-0.8283	-0.0622	-0.1110	**0.0191**	-0.1368	-0.2834
Case 3	0	0.3435	**0.7179**	0.2064	0.2234	-0.9265	-1.0170
Case 4	0	1.1717	**1.5282**	0.6494	0.6984	0.6236	0.5446
ResNet50 NVIDIA Jetson Nano (RN)							
	Base	KD-CNN	KD-LeNet	Pruning	Q-aware	Q-Conv	Q-Dual
Case 1	0	**0.5846**	0.5565	-0.2881	-0.0209	0.0655	-0.0013
Case 2	0	-0.3702	-0.1349	-0.5437	0.0244	**0.0657**	-0.0763
Case 3	0	**1.1239**	1.0066	-0.3206	-0.0870	-0.3951	-0.4535
Case 4	0	**1.5846**	1.3542	-0.2881	-0.0209	0.5915	0.5248

표 3-9, 3-10을 봤을 때, 측정 결과의 점수가 GPU 서버와 Edge Device에서 유사한 동향을 보였다. 표 3-9에서 GPU 서버에서 '양자화 인식 훈련을 마친 VGG19 모델'의 Case 2(정확도가 중요할 때)를 제외한 모든 Case에서 '지식 증류를 적용한 LeNet 모델로 압축을 진행'한 경우의 점수가 가장 높은 것을 볼 수 있다.

Edge Device에서의 점수를 계산한 표 3-10을 봤을 때, 베이스 모델이 VGG19인 경우 '양자화 인식 훈련을 마친 VGG19 모델'의 Case 2를 제외하고는 '지식 증류를 적용한 LeNet 모델로 압축을 진행'했을 시, 모든 Case의 점수가 제일 높았다. 베이스 모델이 ResNet50인 경우, Case 2(정확도가 중요할 때)에서는 '훈련 후 양자화를 적용한 ResNet50 모델'이 가장 높은 Score를 보였으나, 다른 모든 Case에서는 '지식 증류를 적용한 CNN모델로 압축'을 했을 때 가장 점수가 높았다.

표 3-9, 표 3-10과 지금까지의 실험결과에 따르면, 식 (11)에 따라 모든 측정치는 같은 스케일을 갖는다. 따라서 전부 같은 가중치를 가지는 Case 1의 경우, Case 2는 좋지 않은 Score를 보였으나, Case 3, Case 4에서 가장 높은 Score를 받은 지식 증류가 다른 두 가지 모델 압축 기법보다 전반적으로 우수한 Score를 받았다.

딥러닝 모델들을 컴퓨팅 자원이 충분하지 않은 IoT/Edge Device 환경에 적용하기 위해 모델 압축(Model Compression) 기법의 필요성이 증가하고 있다. 정확도, 처리 시간, 파일 크기 같은 우선순위에 특화된 모델 압축 기법은 실험을 통해 전반적으로 지식 증류인 것을 확인하였다. 따라서 지식 증류 기법을 활용하여 본 연구의 핵심인 성능이 우수하며 가벼운 모델에 대해서 5장에서 연구를 이어간다.

제 4 장 데이터셋

4.1 Pothole 데이터 소개

본 연구에서는 자율 주행 시스템에서 도로 노면 상황을 판단하기 위한 가정하에 Pothole 데이터셋 [16]을 활용한다. Pothole 데이터셋은 도로에 대한 "Normal" 클래스와 도로 파손 및 균열이 있는 "Potholes" 클래스로 구성되어 있으며, 각 이미지의 크기는 다양하다. 아래 표 4-1은 Pothole 데이터셋의 정보이며, Pothole 데이터셋은 Normal 클래스의 이미지 352개, Pothole 클래스 이미지 329개로 이루어져 있다.

본 연구의 목표는 적은 양의 데이터로도 우수한 성능을 가진 모델을 개발하는 것입니다. 그러나 기존의 Pothole 데이터셋은 매우 제한적인 양의 데이터를 포함하고 있

다. 따라서 우리는 다음 절에서 데이터의 증분과 전처리를 통해 데이터셋의 다양성을 높이고 모델의 학습 성능을 향상시킨다.

표 4-1 Pothole 데이터셋 정보

	Pothole DataSet
Num of Classes	2
Num of data	681 (Normal: 352, Potholes: 329)
Size of Image	Variety

4.2 데이터 전처리와 데이터 증분

딥러닝 모델의 학습에 있어 데이터의 전처리와 데이터 증분의 과정은 매우 중요하다. 우리가 사용하는 데이터는 이미지 데이터이며, 이미지 데이터 전처리는 모델 학습의 성능과 일반화 능력을 향상시키기 위해 중요한 역할을 한다.

이미지 데이터의 전처리의 효과로 첫 번째는 데이터 일관성 보장을 보장한다. 입력 이미지를 일관된 형식으로 조정하여 모델이 예상하는 형식에 맞게 만든다. 일관된 입력 형식은 모델이 데이터를 올바르게 이해하고 처리할 수 있도록 한다. 입력 데이터의 일관성으로 이미지의 크기가 있으며, 딥러닝 모델이 고정된 크기의 이미지를 요구하는 경우, 이미지 크기를 조정하여 일관된 크기로 맞추어야 한다.

두 번째 효과로 데이터 품질 향상이다. 이미지 데이터에는 다양한 요인에 의해 잡음이나 불필요한 정보가 포함될 수 있다. 전처리 단계에서 이미지의 품질을 향상시키기 위해 필터링, 노이즈 제거, 경계 강조 등의 기법을 사용할 수 있으며, 이를 통해 모델이 이미지에서 핵심적인 정보를 추출하고 불필요한 잡음을 배제할 수 있다. 이미지 데이터 전처리는 모델의 학습과 예측 과정에서 데이터의 품질, 일관성 등을 향상시키는 역할을 하며, 효과적인 전처리는 모델의 성능을 향상시킨다.

또한 딥러닝 모델의 학습에 있어 데이터 증분(Data Augmentation) 작업은 모델 학습에 있어서 매우 중요한 역할을 수행한다 [17]-[19]. 데이터 증분은 기존의 데이터

를 변형하거나 새로운 데이터를 합성하여 데이터셋을 확장하는 과정을 말한다. 데이터 증분 작업의 효과로 첫 번째는 데이터 다양성 제공한다. 데이터 증분은 학습 데이터의 다양성을 높여 모델이 다양한 패턴과 변형에 대응할 수 있도록 하며, 이는 모델의 일반화 능력을 향상시키고, 새로운 입력 데이터에 대한 예측 능력을 개선한다. 두 번째 효과로는 과적합(Over Fit)을 방지한다. 모델이 주어진 데이터에 지나치게 특화되어 다른 데이터에 대한 일반화 능력이 낮아지는 현상인 과적합을 방지하는 데 도움을 주며, 데이터 증분은 학습 데이터셋을 확장하므로, 모델이 일부 데이터에 지나치게 의존하는 것을 방지하고 다양한 패턴과 변형에 대해 학습한다. 마지막 효과로 데이터 부족 문제를 해결한다. 데이터의 부족으로 인해 모델이 충분히 학습되지 못하거나 일반화 능력이 제한될 수 있다. 데이터 증분은 기존 데이터를 변형하거나 합성하여 데이터셋을 확장하므로, 적은 양의 데이터로도 효과적인 학습을 수행할 수 있다.

따라서 우리의 연구에서 기존의 Pothole 데이터셋을 전처리 과정에서 이미지 데이터의 정규화와 이미지 사이즈를 통일 시킨다. 또한 그림 4-1과 같이 이미지 별로 랜덤 플립(Random Flip), 가우시안 블러(Gaussian Blur) 및 밝기 조절을 적용하여 데이터를 증분한다. 데이터 전처리와 증분과정을 끝낸 Pothole 데이터의 정보는 다음 표 4-2와 같다.

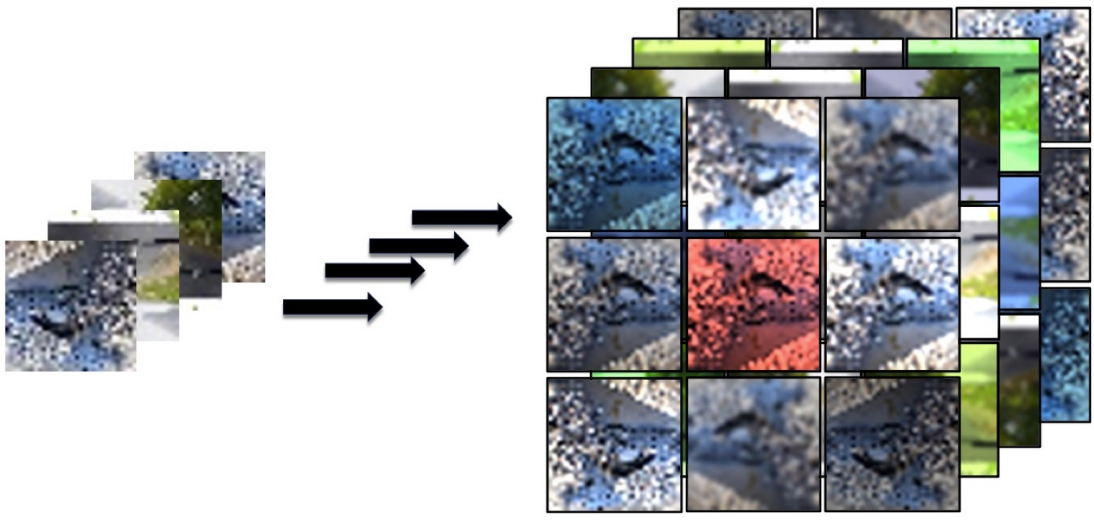


그림 4-1 Pothole 데이터의 증분 작업 도식화

표 4-2 데이터 전처리 및 증분 후의 Pothole 데이터셋 정보

	Pothole DataSet	
Num of Classes	2	
Num of data	Training	Normal: 3,080 Pothole: 2,893
	Validation	Normal: 396 Pothole: 363
	Test	Normal: 385 Pothole: 363
	Total	Normal: 3,861 Pothole: 3,619
Size of Image	32x32	

제 5 장 SDiT 모델 제안 및 Hyperparameter 튜닝

T. Li 등의 최근 보고서 [20], O. Kotevska 등의 연구 [21], 그리고 M. Hagan 등의 연구 [22]에 따르면, IoT/Edge Device의 증가로 인해 데이터 개인 정보 보호에 대한 규정이 국제적으로 이루어지고 있다. 이러한 Edge Device를 사용하여 사용자의 데이터를 이용하여 AI 모델을 학습할 때, 모든 데이터를 중앙에서 수집하고 학습하는 데이터 중앙 학습은 사용자의 데이터가 네트워크를 통해 이동하므로 보안에 취약하며 데이터 개인 정보 보호 문제를 야기할 수 있다. 따라서 우리의 연구에서는 사용자의 데이터를 중앙으로 전송하는 대신 엣지 디바이스 각각에서 사용자의 데이터와 같이 작은 양의 데이터를 사용해서 효과적인 학습을 할 수 있는 모델에 대해서 연구하는 것을 목표로 한다.

5.1 SDiT

기존의 ViT는 JFT-300M과 같은 대량의 데이터로 학습해야 높은 성능을 보인다는 단점이 있었다. 이러한 단점을 극복하기 위해 DeiT 모델이 개발되었으며, DeiT 모델은 ImageNet 데이터만을 사용하여 학습하며, 데이터 증분과 컨볼루션 신경망(CNN)을 Teacher 모델로 사용하여 지식 증류 기법을 적용하여 성능을 향상시켰다. 하지만 DeiT 모델에도 사전에 학습된 CNN 모델이 필요하다는 단점이 있다. 또한, 지식 증류를 위해 CNN 기반의 모델을 Teacher 모델로 사용하려면 Student 모델과 동일한 데이터로 학습되어야 하며, 이를 위해서는 실생활에서는 사용자의 개인 데이터가 네트워크를 통해 중앙으로 수집되어야 가능하다. 이렇게 개인의 데이터가 네트워크를 통해서 중앙으로 모이는 것은 데이터 프라이버시에 큰 위험의 여지가 생긴다.

본 연구에서는 적은 데이터를 사용해서 사전 학습된 다른 모델의 도움 없이 효과적으로 학습하여 높은 성능을 내는 모델에 대해서 연구했다. 기존의 Transformer 기반의 ViT 모델을 자기 지식 증류 기법을 이용하여 학습하는 Self-Knowledge Distillation image Transformer(SDiT) 모델을 제안한다. SDiT의 모델 구조는 다음 그림 5-1과 같다.

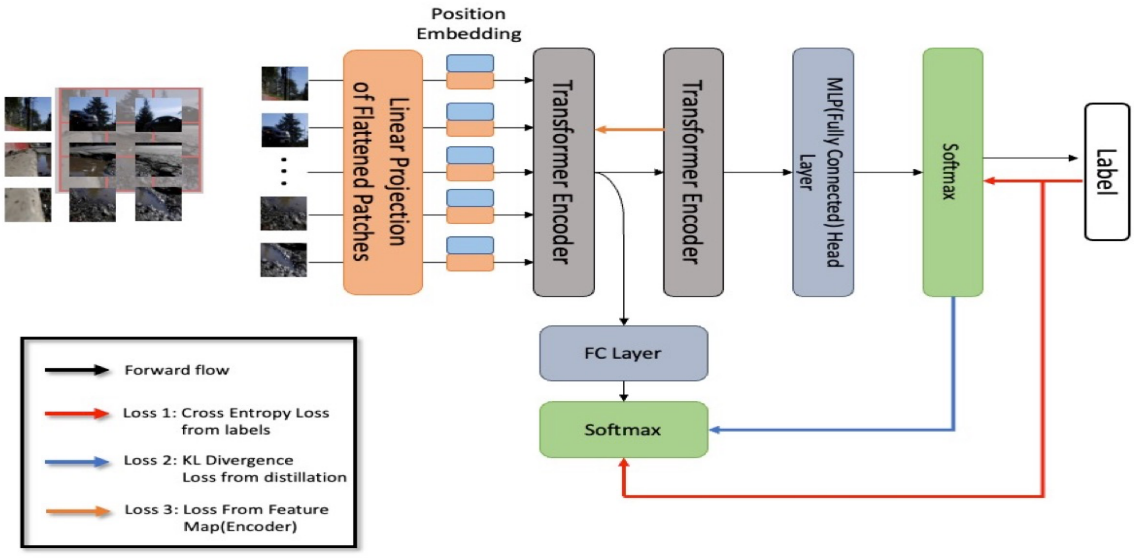


그림 5-1 Self-Knowledge Distillation image Transformer(SDiT) 모델 구조

기존 자기 지식 증류 기법에서는 그림 2-2와 같이 보조 분류기에 병목 블록(Bottleneck Block)을 구성한다. 하지만, 자기 지식 증류에서 병목 블록을 구성한 이유는 파라미터 수 감소 효과와 효율적인 표현 학습 지식 전달 강화를 위해 사용된다.

하지만 SDiT는 Transformer의 인코더(Encoder)를 사용하면서 다양한 표현에 대해서 학습을 진행을 하기에 SDiT의 보조 분류기에서는 추가적인 병목 블록 또는 Transformer 인코더가 추가되지 않고 Fully Connected Layer만으로 이루어진 MLP(Multi Layer Perceptron) 블록을 보조 분류기로 사용한다. 이에 대한 손실 값은 다음 식 (12)와 같이 계산 된다.

$$
\begin{aligned}
L_{TotalWithoutFeature} &= \sum_{i}^{C} loss_i \\
&= \sum_{i}^{C} ((1-\alpha) \bullet CrossEntropy(y^{j}_{Predict}, y_{Truth}) \\
&\quad + \alpha \bullet KLD(y^{j}_{Predict}, y^{C}_{Predict}) \ )
\end{aligned}
\tag{12}
$$

하지만 본 연구에서는 기존의 자기 지식 증류의 보조기의 병목 블록 대신 Transformer의 Encoder의 특징 맵(Feature Map)을 추출하여 구하는 손실도 다음 식 (13)과 같이 구하여 같이 실험을 진행했다.

$$
\begin{aligned}
L^{i}_{FeatEncoder} &= \| F^{i}_{Encoder} - F^{C}_{Encoder} \|^{2}_{2} \\
L_{TotalWithFeature} &= \sum_{i}^{C} loss_i \\
&= \sum_{i}^{C} ((1-\alpha) \bullet CrossEntropy(y^{j}_{Predict}, y_{Truth}) \\
&\quad + \alpha \bullet KLD(y^{j}_{Predict}, y^{C}_{Predict}) \\
&\quad + \lambda \bullet L^{i}_{FeatEncoder})
\end{aligned}
\tag{13}
$$

본 연구에서는 Pothole 데이터셋을 사용하여 실험을 진행하는 것을 고려하여, 기존의 ViT-B/16과 같이 1000개 또는 최소 10개의 Class를 분류하는 모델과 같이

Transformer 인코더 12개를 사용하는 모델은 Class 2개만을 분류하는 모델과는 맞지 않는 것으로 판단되었다. 따라서 본 연구에서는 Transformer 인코더 2개를 사용한 SDiT 모델을 실험에 활용했다. 이를 통해 데이터셋의 특성에 적합한 모델 구조를 선택하여 실험의 효과적인 실험을 진행했다.

5.2 Hyperparameter 튜닝

Hyperparameter는 딥러닝 모델의 학습을 진행 할 때, 사용자가 직접 설정해야 하는 값을 의미하며, 학습률(Learning Rate), 에폭치(Epoch), 배치 사이즈(Batch Size) 등이 있다. 모델의 성능인 정확도를 최대화하기 위해 최적의 Hyperparameter 값을 찾는 과정을 Hyperparameter 튜닝이라고 한다. 본 실험에서는 베이지안 최적화(Bayesian Optimization) [23]를 사용하여 Hyperparameter 튜닝을 진행했다.

베이지안 최적화는 목적 함수의 최적화를 위한 알고리즘이다. 이 알고리즘은 목적 함수에 대한 평가 결과를 사용하여 효율적으로 최적의 매개 변수 값을 찾는 방법이며, 일반적으로, 목적 함수는 입력 매개 변수 세트에 대한 출력 값을 계산하는 함수이다. 베이지안 최적화의 주요 아이디어는 목적 함수에 대한 확률 모델을 구축하고, 이 모델을 사용하여 매개 변수 공간을 탐색하는 한다. 이를 통해 최적의 매개 변수 값을 추정하고, 실제 목적 함수 평가를 수행하여 새로운 데이터를 수집하고, 이 과정을 반복하면서 점진적으로 목적 함수의 최적 값을 찾는다.

제 6 장 SDiT 실험 환경 및 실험 결과 분석

이번 장에서는 본 연구에서 제안하는 SDiT 모델을 Pothole 데이터셋을 사용하여 학습한 결과에 대해서 분석한다. 또한 다른 모델과의 비교를 통해 본 연구에서 제안하는 SDiT 모델에 대해서 검증한다.

6.1 실험 환경

본 연구의 실험에서 GPU 서버에서의 쿠버네티스 환경의 파드에서 실험을 진행했다. GPU 서버의 스펙은 표 5-1과 같으며, GPU 서버 환경은 NVIDIA RTX 2080Ti GPU 8개가 장착되어 있으나 단일 GPU 환경에서 모델의 학습과 추론 측정을 위해, GPU 1개의 자원을 쓸 수 있도록 Pod에서 모델의 학습과 추론 측정을 진행하였다.

표 5-1 GPU Server 실험 환경 스펙 시트

GPU Server		
OS		CentOS 7.9
CPU		Intel(R) Xeon(R) Silver 4210 CPU *2
GPU		RTX 2080Ti * 8
RAM		125 GB
Kubernetes		1.21.10
Pod	**Num of GPU**	RTX 2080 Ti * 1
	Num of CPU Core	4 Core
	RAM	40 GB
	TensorFlow	2.5.1

6.2 실험 결과 및 분석

본 실험에서는 Pothole 데이터셋을 사용하여 SDiT 모델과 다양한 모델에 대해서 학습 후 성능(정확도)을 측정한다. 본 실험에서는 우선적으로 데이터 증강을 적용을 하는 것이 효과적인지 살펴보며, 앞서 설명한 2가지 방법의 손실 값을 이용한 학습의 차이점에 대해서 분석하고 다양한 모델에서 Hyperparameter 튜닝을 적용한 결과를 살펴 본다. 또한 본 실험에서 사용하는 모델은 ViT 기반의 Transformer 인코더 2개를 사용하여 'SDiT-2'라고 지칭하며, 마찬가지로 ViT 또한 Transformer 인코더 2개를 사용하여 'ViT-2'라고 지칭한다.

6.2.1 데이터 증분 검증

본 실험에서 사용하는 Pothole 데이터셋은 총 681개의 이미지로 구성되어 있다. 이는 딥러닝 모델이 학습을 하는데 매우 적은 데이터 이므로 학습의 효과를 올리기 위해 데이터 증분 작업을 통해서 데이터의 수를 늘리는 작업을 진행했다. 해당 실험의 데이터 증분은 이미지를 하나씩 불러와 확률적으로 상하좌우를 뒤집는 Random Horizon Flip, Random Vertical Flip을 적용하며, 또한 가우시안 블러 및 밝기 조절을 진행 하였다. 증분된 데이터와 원본 데이터는 모두 전처리 과정에서 모두 32x32 사이즈의 이미지로 저장했으며, 이미지 정규화 과정을 거쳐 저장되었다.

다음 표 6-1을 통해서 데이터 증분의 효과를 검증한다. 데이터 증분의 검증은 SDiT-2 모델을 사용하여 검증했으며, Hyperparameter 튜닝을 적용하지 않고 100회의 학습과 테스트를 진행하여 테스트 정확도를 측정했다. 측정 결과 SDiT-2 모델에서 원본 데이터(Base)의 평균 정확도는 95.856% 이며, 데이터 증분을 적용했을 때의 평균 테스트 정확도는 97.25%로 측정 되었다. 높은 정확도를 보이는 이유로 우선 데이터의 Class가 2개라는 점에서 Binary 분류 작업이기에 높은 정확도를 보이며, 다른 최대, 최소 정확도로 비교 했을 때, 데이터 증분을 적용한 데이터가 더 높은 수치를 보이며 데이터 증분 작업이 유효했음을 보였다.

표 6-1 데이터 증분 실험 결과 표

	Avg Accuracy	Min Accuracy	Max Accuracy
Base	95.856%	83.82%	98.65%
Adapt Augmentation	97.25%	86.36%	99.96%

6.2.2 SDiT 손실 함수 비교

본 실험에서는 기존의 자기 지식 증류에서 사용되는 손실 값을 기반으로 SDiT에

적용한 손실($L_{TotalWithFeature}$)와 특징 맵을 적용하지 않은 손실($L_{TotalWithoutFeature}$)을 이용하여 성능을 측정한다. 두 손실을 모두 동일하게 SDiT-2에 대해서 측정하며 두 모델 전부 Hyperparameter 튜닝을 하여 해당 결과에 대해서 다음 표 6-2에서 결과를 확인할 수 있다.

표 6-2 손실 함수 비교 실험 결과 표

	SDiT-2(WithoutFeature)		SDiT-2(WithFeature)	
Num of Parameter	Main Classifier	5,554,185		
	Aux Classifier	99,625		
Model Size (MB)	Main Classifier	65MB		
	Aux Classifier	1.8MB		
GPU Memory (MiB)	Main Classifier	435MiB		
	Aux Classifier	307MiB		
Inference Time (ms)	Main Classifier	0.049744652	Main Classifier	0.049743652
	Aux Classifier	0.033504011	Aux Classifier	0.033724011
Evaluate Result	Main Classifier	93.05 %	Main Classifier	94.25 %
	Aux Classifier	92.08 %	Aux Classifier	92.07 %

표 6-2에서 손실 $L_{TotalWithoutFeature}$의 테스트 정확도 결과가 메인 모델에서 93.05%, 보조 분류기의 정확도가 92.08%가 나오며, 손실 $L_{TotalWithFeature}$의 메인 모델 정확도 94.25%, 보조 분류기 정확도 92.07%로 측정 되었다. 이는 오차 범위내의 측정값이며 또한, 보조 분류기의 정확도는 차이가 없음을 보였다. $L_{TotalWithoutFeature}$ 손실 값을 구하는 시간이 $L_{TotalWithFeature}$을 계산하는 시간보다 더 적은 시간이 소요

된다. 따라서 SDiT에서는 $L_{TotalWithoutFeature}$을 사용하는 것이 학습 시간 적인 측면에서 더 효율적이다.

Transformer 인코더의 특징 맵 추출이 기존의 컨볼루션 레이어의 특징 맵을 추출하여 손실을 구하는 것과 차이가 없는 이유를 분석했을 때, 컨볼루션 레이어는 이미지의 지역적인 특징을 추출하지만 Transformer 인코더에서는 이미지 전체에서 일반화된 특징을 추출하는 작업을 진행을 하기에 표 6-2의 결과처럼 큰 차이가 보이지 않는 것으로 분석된다. 이런 인코더의 특징으로 인해 기존의 자기 지식 증류의 보조 분류기처럼 추가적인 컨볼루션 레이어가 필요하지 않는 보조 분류기를 사용할 수 있어 더 적은 파라미터로 자기 지식 증류 기법을 사용할 수 있는 장점을 가진다.

6.2.3 다른 모델과의 비교

본 실험에서 마지막으로 기존의 다른 딥러닝 모델들과 SDiT-2 모델간의 성능을 비교 한다. 본 실험에서 비교하는 모델은 ResNet50, MobileNetV3_Large, EfficientNet_B0, ViT-2이다. ResNet50은 일반적인 컨볼루션 신경망 모델과 SDiT모델의 성능의 창이를 비교하기 위해 선정했으며, MobileNetV3_Large과 EfficientNet_B0는 IoT/Edge Device를 위해 설계된 모델 중 SDiT-2와 파라미터 수가 가장 비슷한 모델로 선정했다. 또한 ViT-2는 기존의 ViT와 SDiT간의 성능의 차이점을 비교하기 위한 비교 군으로 선정했다.

본 실험에서 선정한 모델은 모두 Hyperparameter 튜닝을 적용하여 SDiT와 같은 에폭의 조건에서 학습 되었다. 표 6-3은 Hyperparameter 튜닝이 적용되어 Pothole 데이터셋에 대하여 최적화 된 모델의 결과이다. 각 모델의 추론 타임은 총 11회 측정했으며, 처음 1회는 제외 한 10회 측정 결과의 평균값이다. SDiT-2의 경우 메인 모델의 분류기에서의 추론 속도가 ResNet50, MobileNetV3_Large, EfficientNet_B0보다 빨랐으며, ViT-2와는 오차 범위 내의 속도를 보였다. 이는 모델을 구성하는 레이어의 수에 의해 SDiT-2, ViT-2가 적은 레이어를 사용한 효과로 분석된다.

표 6-3 다른 모델간의 비교 결과 표

	SDiT-2 (WithoutFeature)		ResNet50	MobileNetV3_Large	EfficientNet_B0	ViT-2
Num of Parameter	Main	5,554,185	23,591,810	4,228,994	4,052,133	5,554,185
	Aux	99,625				
Model Size (MB)	Main	65 MB	95 MB	22 MB	21 MB	65 MB
	Aux	1.8 MB				
GPU Memory (MiB)	Main	435 MiB	559 MiB	335 MiB	335 MiB	435 MiB
	Aux	307 MiB				
Inference Time (ms)	Main	0.049744652	0.123386364	0.17075021	0.181264601	0.054410428
	Aux	0.033504011				
Evaluate Result	Main	93.05 %	90.78	88.64	89.71	85.03
	Aux	92.08 %				

또한 정확도 측면에서 SDiT-2가 ViT-2보다 더 높은 정확도를 보였으며, 이를 통해 자기 지식 증류 기법의 적용을 통해 향상된 성능을 보였다고 분석 된다. 다른 ResNet50, MobileNetV3_Large, EfficientNet_B0와 같은 컨볼루션 신경망 기반의 모델들과 비교 했을 때, SDiT의 성능이 더 높게 측정이 되는 이유로 SDiT의 기반이 되는 Transformer의 인코더를 통해서 학습용 데이터에 의존도가 낮고 일반화된 모델로 학습이 되었다고 분석된다.

제 7 장 결론 및 향후 연구

7.1 향후 연구

본 연구의 향후 연구로서 다양한 영역에서 SDiT 모델을 적용을 추구하고자 한다. 본 연구에서는 이미지 분류 영역에서만 적용했으며, 이미지 세그멘테이션, 객체 감지와 같은 다양한 컴퓨터 비전의 영역에 적용을 연구하고자 한다. 또한 적은 데이터만으로 성능을 높일 수 있다는 장점을 통해 의료 영역과 같이 데이터가 제한되며, 수가 적은 영역에도 SDiT 모델의 적용을 통해 높은 성능을 달성할 수 있다고 사료된다.

또한 앞서 설명한 데이터 프라이버시의 규제가 생김에 따라서 그림 7-1과 같이 다수의 Edge Device를 연결한 연합학습과 같은 분산 학습 영역에 적용하여 데이터 프라이버시 규제에 대한 대응으로 제시 할 수 있다고 사료된다.

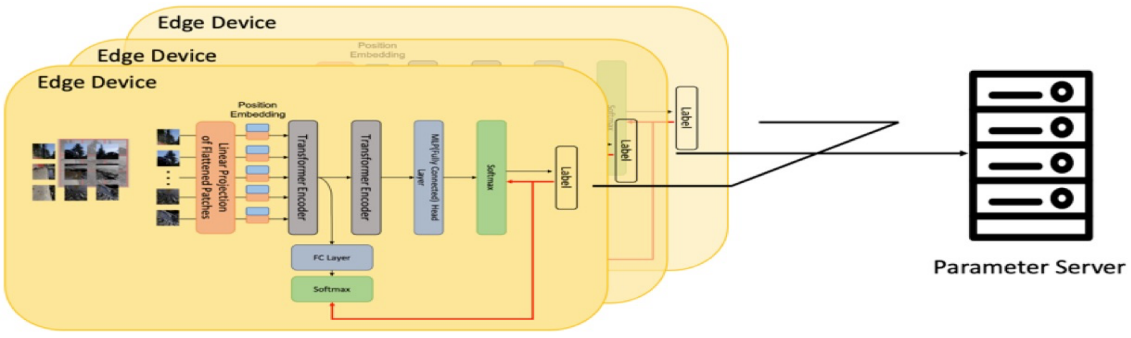


그림 7-1 SDiT 모델의 연합학습 적용 아키텍처

7.2 결론

본 연구에서는 다양한 모델 압축 기법인 지식 증류 기법, 가중치 가지치기, 양자화에 관해서 ImageNet 데이터로 사전 학습된 모델을 CIFAR-10 데이터로 전이 학습을 진행하며, 앞서 소개한 다양한 압축 기법에 대해서 실험하며 정확도, 모델 파일 크기, 추론 속도와 같은 요소들을 중점적으로 분석했다. 실험의 분석 결과, 지식 증류 기법이 가장 우수한 모델 압축 기법임을 확인하였고, 이를 기반으로 Teacher 모델 없이 성능을 향상시킨 자기 지식 증류 기법을 도입하여 최신의 컴퓨터 비전 영역에서 최고 수준의 성능을 보여주는 Transformer 기반의 ViT 모델에 적용하는 SDiT 모델을 제안했다. 본 연구에서 제안한 SDiT 모델은 ImageNet 1K, ImageNet 21K와 같은 다른 대규모 데이터의 사전 학습 없이 Pothole과 같은 적은 데이터만으로도 높은 정확도와 빠른 추론 속도를 보여주며 유의미한 결과를 보였다.

또한 기존의 자기 지식 증류 기법에서 적용되는 특징 맵의 차이를 이용한 손실값이 SDiT에서는 유효하지 않다는 것을 보이며, 모델에 학습 중에 소요되는 연산량을 줄이는 효과도 얻을 수 있었다. 이를 통해서 ViT 모델과 관련된 기존의 진행된 Transformer Encoder와 컨볼루션 레이어를 결합하여 모델의 복잡성을 높여 성능을 높이는 연구 또는 단일 데이터에서 학습 배치에서 다양한 데이터를 입력으로 넣어주는 데이터 증분과 관련된 연구 외에도 비교적 단순한 모델의 변화를 통해서 적은 데이터만을 사용해서 성능을 높이는 결과를 보였다.

참고 문헌

[1] Y.J. Lee, Y.H. Moon, J.Y Park, O.G. Min, "Lightweight Deep Learning Technology Trend," [ETRI] Electronic Communication Trend Analysis, vol. 34, no. 2, pp. 40-50, Apr. 2019.

[2] A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," arXiv:2010.11929 [cs], Jun. 2021, Accessed: Dec. 05, 2021. [Online]. Available: http://arxiv.org/abs/2010.11929

[3] G. E. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," in NIPS Deep Learning Workshop, 2014.

[4] Song Han, Huizi Mao, and William J Dally. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. In ICLR, 2016

[5] Y. Cheng, D. Wang, P. Zhou, and T. Zhang, "Model Compression and Acceleration for Deep Neural Networks: The Principles, Progress, and Challenges," IEEE Signal Processing Magazine, vol. 35, no. 1, pp. 126-136, Jan. 2018, doi: 10.1109/MSP.2017.2765695.

[6] K. Nan, S. Liu, J. Du and H. Liu, "Deep model compression for mobile platforms: A survey," in Tsinghua Science and Technology, vol. 24, no. 6, pp. 677-693, Dec. 2019, doi: 10.26599/TST.2018.9010103.

[7] L. Zhang, J. Song, A. Gao, J. Chen, C. Bao, and K. Ma, "Be Your Own Teacher: Improve the Performance of Convolutional Neural Networks via Self Distillation," presented at the Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019, pp. 3713-3722. Accessed: Sep. 07, 2022. [Online]. Available: https://openaccess.thecvf.com/content_ICCV_2019/html/Zhang_Be_Your_Own_Teacher_Improve_the_Performance_of_Convolutional_Neural_ICCV_2019_paper.htm

I

[8] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning Convolutional Neural Networks for Resource Efficient Inference." arXiv, Jun. 08, 2017. doi: 10.48550/arXiv.1611.06440.

[9] "Quantizing integers after training with int16 activation|TensorFlow Lite." https://www.tensor flow.org/lite/performance/post_training_integer_qu ant_16x8 (accessed April 10, 2021).

[10] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition." arXiv, Apr. 10, 2015. doi: 10.48550/arXiv.1409.1556.

[11] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," presented at the Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770-778. Accessed: May 22, 2023. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html

[12] A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications." arXiv, Apr. 16, 2017. doi: 10.48550/arXiv.1704.04861.

[13] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks." arXiv, Sep. 11, 2020. doi: 10.48550/arXiv.1905.11946.

[14] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jegou, "Training data-efficient image transformers & distillation through attention," in Proceedings of the 38th International Conference on Machine Learning, PMLR, Jul. 2021, pp. 10347-10357. Accessed: Sep. 22, 2021. [Online]. Available: https://proceedings.mlr.press/v139/touvron21a.html

[15] “Smoothing non-critical weights | TensorFlow Model Optimization,” TensorFlow. https://www.tensorflow.org/model_optimization/guide/pruning?hl=ko (accessed April 10, 2021)

[16] “Pothole Detection Dataset.” https://www.kaggle.com/datasets/atulyakumar98/pothole-detection-dataset (accessed May 30, 2023).

[17] T. He, Z. Zhang, H. Zhang, Z. Zhang, J. Xie, and M. Li, “Bag of Tricks for Image Classification with Convolutional Neural Networks.” arXiv, Dec. 05, 2018. doi: 10.48550/arXiv.1812.01187.

[18] E. Hoffer, T. Ben-Nun, I. Hubara, N. Giladi, T. Hoefler, and D. Soudry, “Augment Your Batch: Improving Generalization Through Instance Repetition,” presented at the Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 8129-8138. Accessed: May 03, 2023. [Online]. Available: https://openaccess.thecvf.com/content_CVPR_2020/html/Hoffer_Augment_Your_Batch_Improving_Generalization_Through_Instance_Repetition_CVPR_2020_paper.html

[19] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “mixup: Beyond Empirical Risk Minimization.” arXiv, Apr. 27, 2018. doi: 10.48550/arXiv.1710.09412.

[20] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, “Federated Learning: Challenges, Methods, and Future Directions,” IEEE Signal Processing Magazine, vol. 37, no. 3, pp. 50-60, May 2020, doi: 10.1109/MSP.2020.2975749.

[21] O. Kotevska, J. Johnson, and A. G. Kusne, “Analyzing Data Privacy for Edge Systems,” in 2022 IEEE International Conference on Smart Computing (SMARTCOMP), Jun. 2022, pp. 223-228. doi: 10.1109/SMARTCOMP55677.2022.00058.

[22] M. Hagan, F. Siddiqui, and S. Sezer, "Enhancing Security and Privacy of Next-Generation Edge Computing Technologies," in 2019 17th International Conference on Privacy, Security and Trust (PST), Aug. 2019, pp. 1-5. doi: 10.1109/PST47121.2019.8949052.

[23] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas, "Taking the Human Out of the Loop: A Review of Bayesian Optimization," Proceedings of the IEEE, vol. 104, no. 1, pp. 148-175, Jan. 2016, doi: 10.1109/JPROC.2015.2494218.

ABSTRACT

Improving Model Performance through Deep Learning Model Lightweighting Techniques and Research on the SDiT Model Technology.

Jin-Wook Choi

Department of Software and Communications Engineering

The Graduate School of Hongik University

In recent years, the field of artificial intelligence and deep learning has witnessed continuous advancements in models powered by large datasets and powerful computing capabilities. The accumulation of big data and advancements in hardware have contributed to the scalability and complexity of deep learning models. While these models offer high performance and accuracy, they also come with increased model size and computational requirements.

To address these challenges, model compression techniques have emerged as an important research topic. Model compression aims to transform deep learning models into smaller and more lightweight forms, reducing model

size and improving inference speed. This enables efficient execution of deep learning models even in resource-constrained environments. In this study, we analyze and compare representative model compression techniques such as Knowledge Distillation, Weight Pruning, and Quantization. Among these techniques, Knowledge Distillation, which involves the transfer of knowledge from a Teacher model to a smaller Student model, has shown superior performance by preserving the accuracy and characteristics of the Teacher model.

Furthermore, in recent years, Vision Transformer (ViT) models, which are based on the Transformer architecture and designed to replace traditional convolutional layers in computer vision tasks, have gained attention. ViT models demonstrate excellent performance by leveraging the power of Transformers for processing image data. However, ViT models suffer from drawbacks such as a large number of parameters and high data requirements, resulting in increased model size and training costs.

Therefore, in this study, we propose a new model called SDiT (Self-Knowledge Distillation image Transformer) to overcome the limitations of ViT models and achieve high performance even with limited data.